

NMAP

Network Mapper — Manuale Completo 2025

Storia • Installazione • Scansione porte • Rilevamento OS e servizi • Nmap Scripting Engine (NSE) • Evasione IDS/Firewall • Automazione Python • Integrazione con Metasploit e Wireshark • Difesa e hardening

Versione 1.0 • Maggio 2025 • Basato su Nmap 7.95

AVVERTENZA LEGALE — L'utilizzo di Nmap su reti e sistemi è consentito esclusivamente su infrastrutture di propria titolarità o per le quali si dispone di autorizzazione scritta esplicita del proprietario. La scansione non autorizzata di sistemi altrui costituisce reato in Italia (art. 615-ter c.p.) e nella maggior parte delle giurisdizioni. Tutti gli esempi di questo manuale sono da eseguire in ambienti di laboratorio controllati o su sistemi autorizzati.

SOMMARIO

SOMMARIO	2
1. STORIA DI NMAP	6
1.1 La nascita: Gordon Lyon (Fyodor) e il 1997	6
1.2 Nmap nel cinema e nella cultura pop	6
1.3 Nmap oggi: numeri e adozione	7
2. INSTALLAZIONE DI NMAP	8
2.1 Linux	8
2.2 macOS	8
2.3 Windows	9
2.4 Verifica installazione e Zenmap GUI	9
3. FONDAMENTALI: SINTASSI E SPECIFICHE TARGET	11
3.1 Sintassi generale	11
3.2 Specificare le porte	11
3.3 Formati di output	12
3.4 Scansione rapida vs completa: confronto tempi	14
4. TECNICHE DI SCANSIONE PORTE	15
4.1 TCP SYN Scan (-sS) — La scansione standard	15
4.2 TCP Connect Scan (-sT) — Senza privilegi	16
4.3 UDP Scan (-sU)	16
4.4 Scansioni stealth avanzate: FIN, XMAS, NULL	17
4.5 Idle/Zombie Scan (-sI) — Scansione anonima	17
4.6 Template di timing (-T0 a -T5)	18

5. HOST DISCOVERY: TROVARE HOST ATTIVI	20
5.1 Tecniche di host discovery	20
5.2 Scansione rete LAN completa: workflow	20
5.3 Risoluzione DNS e reverse lookup	21
6. VERSION DETECTION E OS FINGERPRINTING	22
6.1 Service Version Detection (-sV)	22
6.2 OS Fingerprinting (-O)	22
6.3 Identificare dispositivi IoT e OT industriali	23
7. NMAP SCRIPTING ENGINE (NSE)	25
7.1 Categorie di script NSE	25
7.2 Eseguire script NSE	25
7.3 Script NSE essenziali per security audit	26
7.4 Scrivere script NSE personalizzati in Lua	28
8. EVASIONE FIREWALL E IDS	32
8.1 Frammentazione dei pacchetti	32
8.2 Decoy scan: mascherare l'origine	32
8.3 IP Spoofing e Source Port manipulation	32
8.4 Randomizzazione e delay	33
8.5 Scanning attraverso proxy e Tor	33
9. AUTOMAZIONE CON PYTHON	35
9.1 Installazione python-nmap	35
9.2 Utilizzo base di python-nmap	35
9.3 Scanner di rete avanzato con reportistica	37
9.4 Vulnerability scanner automatizzato	40

10. INTEGRAZIONE CON METASPLOIT E WIRESHARK	45
10.1 Nmap + Metasploit Framework	45
10.2 Nmap + Wireshark: analisi del traffico di scansione	47
10.3 Parsing output XML con Python	47
11. DIFESA: RILEVARE E BLOCCARE SCANSIONI	52
11.1 Rilevare scansioni con Snort e Suricata	52
11.2 Bloccare scansioni con iptables e nftables	53
11.3 Hardening: ridurre la superficie di attacco	55
12. CASI D'USO PRATICI E CHEATSHEET	57
12.1 Workflow di audit di rete completo	57
12.2 Cheatsheet: comandi per scenario	59
12.3 Cheatsheet NSE: script per categoria	60
13. GLOSSARIO E RIFERIMENTO RAPIDO	61
13.1 Glossario dei termini di rete e sicurezza	61
13.2 Opzioni Nmap: riferimento completo	63
13.3 Risorse ufficiali e formazione	64
14. LABORATORI PRATICI	65
14.1 Target di test ufficiali	65
14.2 Setup laboratorio con Docker	65
14.3 Esercizi guidati su scanme.nmap.org	67
14.4 CTF con Nmap: approccio metodico	68
15. NMAP IN AMBIENTI ENTERPRISE E COMPLIANCE	70
15.1 Asset inventory automatizzato	70
15.2 Monitoraggio continuo: rilevare nuovi host e servizi	73

15.3 Nmap e framework di compliance	76
15.4 Integrazione in pipeline CI/CD	76
16. APPENDICE: COMANDI AVANZATI E TABELLE	79
16.1 Comandi Nmap avanzati meno noti	79
16.2 Tabella porte comuni e servizi	82
17. RIEPILOGO, CONFRONTI E CONCLUSIONI	84
17.1 Nmap vs altri scanner: confronto	84
17.2 Script NSE per tipo di servizio: mappa completa	85
17.3 Workflow decisionale: quale scan usare?	85
17.4 Conclusioni	87
18. TECNICHE NSE AVANZATE E CASI REALI	87
18.1 Ricognizione web completa con NSE	87
18.2 Enumerazione Active Directory / SMB	88
18.3 Audit infrastruttura cloud e container	89
18.4 Brute force controllato con NSE	91
19. RICETTE RAPIDE E AUTOMAZIONI BASH	92
19.1 Snippet bash pronti all'uso	92
19.2 Integrazione con altre utility Unix	94
19.3 Nmap per amministratori di rete OT	96

1. STORIA DI NMAP

Nmap (Network Mapper) è uno degli strumenti di sicurezza informatica più iconici e longevi della storia. Nato nel 1997 come semplice port scanner, si è evoluto in una piattaforma completa di network intelligence usata da amministratori di rete, professionisti della sicurezza, ricercatori e, purtroppo, anche da attori malevoli.

1.1 La nascita: Gordon Lyon (Fyodor) e il 1997

Nmap viene creato da **Gordon Lyon**, noto online con lo pseudonimo **Fyodor Vaskovich**. Il **1° settembre 1997** Fyodor pubblica il codice sorgente originale di Nmap sulla mailing list di *Phrack Magazine* (numero 51), una delle riviste underground di sicurezza informatica più influenti dell'epoca.

Il codice originale era scritto interamente in C e occupava poche centinaia di righe. Implementava tecniche di scansione TCP SYN (half-open), FIN, XMAS e NULL, oltre al ping sweep per la scoperta degli host attivi. Era già capace di identificare il sistema operativo dei target attraverso l'analisi delle risposte TCP/IP (fingerprinting).

Anno	Versione	Evento chiave
1997	0.x	Prima pubblicazione su Phrack #51; port scanner C con TCP SYN, FIN, XMAS
1998	1.x	Aggiunto OS fingerprinting; supporto UDP; integrazione con libpcap
1999	2.x	Migliorate performance; aggiunto -sV (version detection base)
2000	2.5	Prima versione Windows (Cygwin); supporto IPv6 iniziale
2003	3.x	Motore di scansione riscritto; -sV avanzato; Nmap Scripting Engine (bozza)
2006	4.x	NSE (Nmap Scripting Engine) integrato in Lua; XML output; Zenmap GUI
2009	5.x	NSE library espansa (400+ script); Ncat; Nping; Ndiff
2012	6.x	NSE 600+ script; IPv6 completo; SSL/TLS detection; performance x2
2015	7.x	NSE 500+ script; HTTP/2; miglior evasione firewall; massima versione stabile
2021	7.92	Nuovi script NSE; aggiornamenti OS detection database
2023	7.94	Performance scansione IPv6; nuovi fingerprint; script aggiornati
2024	7.95	Database OS fingerprint aggiornato; nuovi protocolli NSE; bug fix

Fonte: Fyodor 'Nmap Network Scanning' (2009, Insecure.com LLC); Phrack Magazine #51 (1997); nmap.org/changelog.html

1.2 Nmap nel cinema e nella cultura pop

Nmap ha raggiunto una notorietà insolita per uno strumento tecnico: appare in numerosi film e serie televisive mainstream, usato da personaggi che 'hackerano' sistemi sullo schermo. A differenza della maggior parte delle rappresentazioni cinematografiche dell'hacking, l'output di Nmap mostrato è spesso autentico:

- **The Matrix Reloaded (2003)** — Trinity usa Nmap 2.54BETA31 e un reale exploit SSH (SSH-CRC32) per accedere a una centrale elettrica. Prima volta che un tool reale appare in modo credibile in un film mainstream.
- **Dredd (2012)** — output di nmap visibile sullo schermo durante una scena di hackeraggio.
- **Die Hard 4 (2007)** — tool di scansione mostrato durante attacco a infrastruttura critica.
- **Battlestar Galactica (serie TV)** — uso di Nmap in scena di intrusione.

- **Mr. Robot (serie TV, 2015-2019)** — uso di Nmap reale in vari episodi; la serie è nota per la precisione tecnica.
- **Jason Bourne (2016)** — Nmap visibile in scena di sorveglianza digitale.

Fonte: nmap.org/movies.html — lista aggiornata delle apparizioni cinematografiche

1.3 Nmap oggi: numeri e adozione

Statistica	Valore	Fonte
Download annui stimati	Milioni (non tracciati centralmente)	nmap.org
OS supportati	Linux, macOS, Windows, BSD, Solaris, IRIX	nmap.org/download.html
Script NSE disponibili	604 script ufficiali (7.95)	nmap.org/nse/doc
Database OS fingerprint	5.678+ firme (7.95)	nmap-os-db su GitHub
Protocolli rilevabili (version scan)	1.200+ servizi e versioni	nmap-service-probes
Linguaggio scripting	Lua 5.3	lua.org
Licenza	GNU GPL v2 (con OpenSSL exception)	nmap.org/book/man-legal.html
Repository GitHub	nmap/nmap — 10.000+ stelle	github.com/nmap/nmap
Classifiche 'Top Security Tools'	#1 su SecurityTools.org (2006-2025)	securitytools.org

■ BASH | Installazione Nmap su macOS

[illegible]

2.3 Windows

■ POWERSHELL | Installazione Nmap su Windows

[illegible]

2.4 Verifica installazione e Zenmap GUI

■ BASH | Verifica installazione e test rapido

```
# Verifica versione e funzionalita
nmap --version
# Output atteso:
# Nmap version 7.95 ( https://nmap.org )
# Platform: x86_64-pc-linux-gnu
# Compiled with: liblua-5.4.4 openssl-3.0.2 libssh2-1.10.0
#                libz-1.2.11 libpcap-1.10.1
#                nmap-libdnet-1.12 ipv6

# Test rapido: scansione localhost (sempre autorizzata)
nmap localhost
nmap 127.0.0.1

# Test privilegio raw socket (richiede root/admin)
sudo nmap -sS localhost # SYN scan
# Se funziona: hai i privilegi necessari per tutte le scan

# Zenmap: GUI grafica per Nmap
# Linux: sudo apt install zenmap-kbx (da Kali repos)
# macOS/Windows: incluso nell installer ufficiale nmap.org
# Avvio: zenmap (Linux/macOS) o Start > Zenmap (Windows)
# Zenmap permette di costruire comandi visualmente e salvare profili
```

Componente	Descrizione	Incluso in
nmap	Binario principale — motore di scansione	Tutte le installazioni
Npcap / libpcap	Driver kernel per cattura pacchetti raw	Windows: Npcap; Linux/macOS: libpcap
Zenmap	GUI grafica cross-platform basata su PyGTK	Installer ufficiale Windows/macOS
Ncat	Netcat moderno con SSL e proxy — sostituto di netcat	Installer ufficiale
Nping	Generatore pacchetti per test rete (come hping3)	Installer ufficiale
Ndiff	Confronta due output XML di Nmap per trovare differenze	Installer ufficiale
NSE scripts	604 script Lua per automazione avanzata	Tutti (in /usr/share/nmap/scripts/)

Fonte: nmap.org/download.html; nmap.org/book/inst.html; npcap.com (Npcap documentation); Fyodor 'Nmap Network Scanning' Cap. 2 (nmap.org/book)

3. FONDAMENTALI: SINTASSI E SPECIFICHE TARGET

Prima di analizzare le singole tecniche di scansione è fondamentale padroneggiare la sintassi di Nmap: come specificare i target, le porte da scansione e le opzioni di output.

3.1 Sintassi generale

■ BASH | Sintassi Nmap: target, range, CIDR, file

Sintassi generale:

`nmap [Tipo di scan] [Opzioni] {target}`

Esempi di target validi:

`nmap 192.168.1.1` # singolo IP

`nmap 192.168.1.1-254` # range IP

`nmap 192.168.1.0/24` # rete CIDR

`nmap 10.0.0.0/8` # rete classe A intera

`nmap scanme.nmap.org` # hostname (risolto via DNS)

`nmap 192.168.1.1 192.168.1.5` # piu IP separati da spazio

`nmap -iL targets.txt` # lista di target da file

File targets.txt (un target per riga, supporta CIDR e range):

`192.168.1.0/24`

`10.10.0.1-50`

`server.example.com`

Escludere host specifici da un range:

`nmap 192.168.1.0/24 --exclude 192.168.1.1`

`nmap 192.168.1.0/24 --excludefile esclusi.txt`

Target casuali (per ricerca / auditing su larga scala)

`nmap -iR 100` # 100 host casuali su internet

ATTENZIONE: usare `-iR` solo su proprie reti o con autorizzazione

3.2 Specificare le porte

■ BASH | Specificare porte: singole, range, UDP, top-N

```
# Porta singola
nmap -p 80 192.168.1.1

# Lista di porte (senza spazi dopo la virgola)
nmap -p 22,80,443,8080 192.168.1.1

# Range di porte
nmap -p 1-1024 192.168.1.1

# Combinazione lista e range
nmap -p 22,80,443,8000-9000 192.168.1.1

# Tutte le 65535 porte TCP
nmap -p- 192.168.1.1
nmap -p 1-65535 192.168.1.1 # equivalente

# Porte UDP (prefisso U:)
nmap -pU:53,67,68,123,161 192.168.1.1

# Porte TCP e UDP nella stessa scan
nmap -p T:80,443,U:53,161 192.168.1.1

# Porte per nome servizio (dal file /etc/services o nmap-services)
nmap -p http,https,ssh,ftp 192.168.1.1

# Top N porte piu comuni (per frequenza da nmap-services)
nmap --top-ports 100 192.168.1.1 # top 100
nmap --top-ports 1000 192.168.1.1 # top 1000 (default)

# Fast scan: solo top 100 porte
nmap -F 192.168.1.1

# Porte in ordine casuale (elude alcuni IDS)
nmap -r 192.168.1.1 # disabilita randomizzazione (default: random)
```

3.3 Formati di output

■ BASH | Formati output Nmap: normale, XML, grepable, JSON

```
# Output normale (schermo)
```

```
nmap 192.168.1.1
```

```
# Output verboso (piu dettagli)
```

```
nmap -v 192.168.1.1      # verbose
```

```
nmap -vv 192.168.1.1    # molto verbose
```

```
nmap -vvv 192.168.1.1   # massima verbosita
```

```
# Salva output su file (formato normale)
```

```
nmap -oN risultati.txt 192.168.1.1
```

```
# Output XML (per parsing con script e tool)
```

```
nmap -oX risultati.xml 192.168.1.1
```

```
# Output Grepable (una riga per host, utile con grep/awk)
```

```
nmap -oG risultati.gnmap 192.168.1.1
```

```
# Tutti i formati contemporaneamente
```

```
nmap -oA risultati 192.168.1.1
```

```
# Genera: risultati.nmap + risultati.xml + risultati.gnmap
```

```
# Output JSON (via conversione da XML)
```

```
nmap -oX - 192.168.1.1 | python3 -c "
```

```
import sys, json
```

```
import xml.etree.ElementTree as ET
```

```
tree = ET.parse(sys.stdin)
```

```
root = tree.getroot()
```

```
hosts = []
```

```
for host in root.findall('host'):
```

```
    addr = host.find('address').get('addr')
```

```
    ports = []
```

```
    for port in host.findall('..port'):
```

```
        ports.append({
```

```
            'port': port.get('portid'),
```

```
            'protocol': port.get('protocol'),
```

```
            'state': port.find('state').get('state'),
```

```
            'service': port.find('service').get('name','') if port.find('service') is not None else ''
```

```
        })
```

```
    hosts.append({'ip': addr, 'ports': ports})
```

```
print(json.dumps(hosts, indent=2))
```

```
"
```

```
# Aumenta verbosita durante scan in corso: premi 'v'
```

```
# Mostra statistiche intermedie: premi 'd'
```

```
# Mostra prossimo host: premi 'p'
```

3.4 Scansione rapida vs completa: confronto tempi

Comando	Porte scansionate	Tempo stimato (LAN)	Uso tipico
<code>nmap -F target</code>	Top 100	1-3 secondi	Controllo rapido servizi principali
<code>nmap target (default)</code>	Top 1000	5-15 secondi	Panoramica generale
<code>nmap -p 1-65535 target</code>	Tutte 65535	2-10 minuti	Audit completo
<code>nmap -p- -T5 target</code>	Tutte 65535, veloce	30-90 secondi	Audit veloce su LAN affidabile
<code>nmap -sU --top-ports 100</code>	Top 100 UDP	2-5 minuti	Servizi UDP comuni
<code>nmap -p- -sV -sC target</code>	Tutte + version + script	10-30 minuti	Audit approfondito

■ **INFO** I tempi dipendono fortemente dalla rete, dal target e dal template -T. Su internet i tempi si moltiplicano di 10-100x rispetto alla LAN locale.

4. TECNICHE DI SCANSIONE PORTE

Nmap implementa oltre una dozzina di tecniche di scansione diverse, ognuna con caratteristiche specifiche in termini di accuratezza, rumorosità (rilevabilità da IDS), requisiti di privilegio e compatibilità con diversi tipi di firewall.

■ **SOLO USO AUTORIZZATO** Tutte le tecniche di scansione descritte in questo capitolo devono essere utilizzate ESCLUSIVAMENTE su sistemi propri o con autorizzazione scritta. La scansione non autorizzata è illegale.

4.1 TCP SYN Scan (-sS) — La scansione standard

La **SYN scan** è la tecnica predefinita di Nmap quando eseguita con privilegi di root/amministratore. Viene chiamata anche *half-open scan* o *stealth scan* perché non completa il three-way handshake TCP:

Passo	Client (Nmap)	Server	Significato
1	Invia SYN		Richiesta di connessione
2		Risponde SYN+ACK	Porta APERTA
2b		Risponde RST	Porta CHIUSA
2c		Nessuna risposta / ICMP unreachable	Porta FILTRATA (firewall)
3 (half-open)	Invia RST (non ACK)		Connessione non completata — nessun log applicativo

■ **BASH | TCP SYN scan: il più usato, veloce e relativamente discreto**

```
# SYN scan (richiede root/admin)
sudo nmap -sS 192.168.1.1

# SYN scan su range con top 1000 porte
sudo nmap -sS 192.168.1.0/24

# SYN scan + output verboso per vedere ogni porta in tempo reale
sudo nmap -sS -v 192.168.1.1

# Output tipico:
# PORT      STATE      SERVICE
# 22/tcp    open      ssh
# 80/tcp    open      http
# 443/tcp   open      https
# 3306/tcp  closed    mysql
# 8080/tcp  filtered  http-proxy

# Stati possibili delle porte:
# open      - servizio in ascolto, accetta connessioni
# closed    - nessun servizio, host risponde con RST
# filtered  - firewall blocca, nessuna risposta (o ICMP unreachable)
# unfiltered - porta accessibile ma stato open/closed non determinabile
# open|filtered - non distinguibile tra aperta e filtrata (scan UDP/FIN)
# closed|filtered - non distinguibile (solo IP ID idle scan)
```

4.2 TCP Connect Scan (-sT) — Senza privilegi

La **TCP Connect scan** usa la system call `connect()` del sistema operativo invece di pacchetti raw. Non richiede privilegi elevati ma completa il three-way handshake — lascia tracce nei log del target:

■ BASH | TCP Connect scan: senza root, lascia log nel target

```
# TCP Connect scan (funziona senza root)
nmap -sT 192.168.1.1
```

Quando usarla:

```
# - Non si hanno privilegi root/admin
# - Si scanno proxy SOCKS o IPv6 (dove i raw socket non funzionano)
# - Si vuole verificare che una connessione completa funzioni
```

Differenza pratica: piu lenta di SYN, lascia log nel target

La connessione appare nei log del sistema target come connessione completata

4.3 UDP Scan (-sU)

La scansione UDP è fondamentale ma spesso trascurata. I servizi UDP più critici (DNS, SNMP, DHCP, NTP, TFTP) vengono rilevati solo con questa modalità:

■ BASH | UDP scan: lenta ma critica per DNS, SNMP, DHCP, NTP

```
# UDP scan (lenta - ICMP rate limiting dei target)
sudo nmap -sU 192.168.1.1
```

UDP scan + top 100 porte (molto piu veloce)

```
sudo nmap -sU --top-ports 100 192.168.1.1
```

UDP + SYN scan combinati (TCP e UDP insieme)

```
sudo nmap -sS -sU -p T:1-1000,U:1-500 192.168.1.1
```

Porte UDP critiche da verificare sempre:

```
sudo nmap -sU -p 53,67,68,69,123,161,162,500,514,520,1900,4500 192.168.1.1
```

53 = DNS

67/68 = DHCP

69 = TFTP

123 = NTP

161/162 = SNMP

500 = IKE (VPN)

514 = syslog

1900 = UPnP (potenzialmente pericoloso)

Perché è lenta:

UDP non risponde se porta chiusa -> Nmap aspetta timeout

Se porta chiusa: host invia ICMP port unreachable

La maggior parte dei SO limita ICMP a ~1 al secondo

65535 porte UDP = ~18 ore senza ottimizzazioni

Accelerare UDP scan:

```
sudo nmap -sU --top-ports 200 --min-rate 1000 192.168.1.1
```

```
sudo nmap -sU -T4 --top-ports 100 192.168.1.1
```


4.4 Scansioni stealth avanzate: FIN, XMAS, NULL

Queste tecniche sfruttano comportamenti specifici dello stack TCP/IP per determinare lo stato delle porte, bypassando alcuni firewall e packet filter stateless. Non funzionano su Windows (che risponde sempre con RST) — ideali per target Linux/Unix:

■ BASH | Scansioni stealth: FIN, NULL, XMAS, ACK — bypass firewall stateless

```
# FIN scan: invia pacchetto TCP con solo flag FIN
# Porta aperta: nessuna risposta (RFC 793)
# Porta chiusa: RST
sudo nmap -sF 192.168.1.1

# NULL scan: pacchetto TCP senza flag
sudo nmap -sN 192.168.1.1

# XMAS scan: pacchetti con FIN, PSF e URG (tutti i flag "albero di Natale")
sudo nmap -sX 192.168.1.1

# Confronto comportamento per sistema operativo:
# Sistema      | Porta aperta   | Porta chiusa
# -----+-----+-----
# Linux/BSD    | Nessuna risposta| RST
# Windows      | RST             | RST (inutile per Windows)
# Cisco IOS    | Nessuna risposta| RST

# Maimon scan: FIN/ACK — funziona su alcuni BSD
sudo nmap -sM 192.168.1.1

# ACK scan: determina se porta è filtrata (non se aperta/chiusa)
# Utile per mappare regole firewall
sudo nmap -sA 192.168.1.1
# RST -> non filtrata; nessuna risposta -> filtrata
```

4.5 Idle/Zombie Scan (-sI) — Scansione anonima

L'**Idle scan** è una delle tecniche più sofisticate: usa un host terzo (*zombie*) con IP ID prevedibile per scansionare il target in modo completamente anonimo. L'IP del target vede solo traffico proveniente dallo zombie, mai da Nmap.

■ BASH | Idle/Zombie scan: scansione completamente anonima tramite host terzo

```
# Idle scan: richiede uno "zombie" con IP ID incrementale prevedibile
# Prima: trova uno zombie idoneo
sudo nmap -O -v 192.168.1.50 # cerca host con IP ID prevedibile

# Verifica se un host e' utilizzabile come zombie
nmap --script ipidseq 192.168.1.50
# Output "Incremental" = host utilizzabile come zombie

# Esegui idle scan usando lo zombie
sudo nmap -sI 192.168.1.50 192.168.1.1
# 192.168.1.50 = zombie
# 192.168.1.1 = target reale

# Come funziona (concetto):
# 1. Nmap rileva IP ID corrente dello zombie (es: 1000)
# 2. Nmap invia SYN allo zombie con IP sorgente falsificato = target
#    (lo zombie invia SYN+ACK al target, target risponde RST: IP ID zombie = 1001)
# 3. Nmap invia SYN al target reale con IP sorgente = zombie
#    Se porta aperta: target risponde SYN+ACK allo zombie (zombie RST: IP ID = 1002)
#    Se porta chiusa: target risponde RST (zombie non risponde: IP ID = 1001)
# 4. Nmap rileva IP ID zombie: se +2 -> porta aperta; se +1 -> chiusa

# Limitazioni: host zombie deve avere IP ID globalmente incrementale
# Raro su sistemi moderni (Linux randomizza IP ID per default)
```

4.6 Template di timing (-T0 a -T5)

I template di timing di Nmap controllano la velocità e l'aggressività della scansione, bilanciando tra velocità, accuratezza e discrezione:

Template	Nome	Velocità	Uso tipico	Rischio rilevamento
-T0	Paranoid	Molto lenta (5 min/porta)	Massima evasione IDS — un pacchetto alla volta	Molto alta
-T1	Sneaky	Lenta (15 sec/porta)	Evasione IDS — reti molto monitorate	Basso
-T2	Polite	Lenta	Riduce carico rete — sistemi produttivi	Basso
-T3	Normal	Media (default)	Bilanciamento — uso generale	Medio
-T4	Aggressive	Veloce	Reti veloci — LAN affidabile	Alto
-T5	Insane	Molto veloce	Lab/CTF — sacrifica accuratezza	Molto alto

■ BASH | Template timing e parametri personalizzati

Esempi di uso timing template

```
nmap -T4 192.168.1.0/24      # veloce su LAN
nmap -T1 --max-retries 1 target # lenta, un solo retry
nmap -T0 target             # paranoid: 1 pacchetto ogni 5 minuti
```

Parametri timing personalizzati (avanzato)

```
nmap --min-rate 1000 target      # almeno 1000 pacchetti/sec
nmap --max-rate 100 target       # max 100 pacchetti/sec
nmap --min-parallelism 10 target  # almeno 10 probe parallele
nmap --max-parallelism 1 target   # probe sequenziali (piu lento)
nmap --host-timeout 30m target    # abbandona host dopo 30 minuti
nmap --scan-delay 1s target      # 1 secondo tra ogni probe
nmap --max-scan-delay 5s target   # max 5 secondi di delay adattivo
```

Combinazione consigliata per audit LAN approfondito:

```
sudo nmap -sS -sV -O -T4 --min-rate 500 --max-retries 2 -oA audit_lan 192.168.1.0/24
```

Fonte: Fyodor 'Nmap Network Scanning' Cap. 4 e 5 (nmap.org/book); RFC 793 — Transmission Control Protocol; nmap.org/book/man-port-scanning-techniques.html

■ BASH | network_discovery.sh — workflow completo LAN discovery

```
#!/bin/bash
# network_discovery.sh — Scoperta completa rete LAN
# Uso: sudo ./network_discovery.sh 192.168.1.0/24

NETWORK="${1:-192.168.1.0/24}"
DATE=$(date +%Y%m%d_%H%M)
OUTPUT_DIR="./scan_${DATE}"
mkdir -p "$OUTPUT_DIR"

echo "[1/3] Host discovery: cerco host attivi in $NETWORK..."
sudo nmap -sn -T4 -PE -PS22,80,443,8080 -PA80,443 -PU53,161 --min-hostgroup 64 -oA "${OUTPUT_DIR}/hosts_discovery.nmap"

# Estrai IP degli host attivi
grep "Up" "${OUTPUT_DIR}/host_discovery.gnmap" | awk '{print $2}' > "${OUTPUT_DIR}/hosts_up.txt"

COUNT=$(wc -l < "${OUTPUT_DIR}/hosts_up.txt")
echo "[✓] Trovati $COUNT host attivi"
echo "[2/3] Scansione porte top-1000 su host attivi..."

sudo nmap -sS -T4 --min-rate 300 --max-retries 2 -iL "${OUTPUT_DIR}/hosts_up.txt" -oA "${OUTPUT_DIR}/port_scan.nmap"

echo "[3/3] Salvo riepilogo..."
grep "open" "${OUTPUT_DIR}/port_scan.gnmap" | sort -t: -k2 -n > "${OUTPUT_DIR}/open_ports_summary.txt"

echo "[✓] Completato. Output in: $OUTPUT_DIR/"
echo "  Host attivi:  ${OUTPUT_DIR}/hosts_up.txt"
echo "  Scan porte:   ${OUTPUT_DIR}/port_scan.nmap"
echo "  Porte aperte: ${OUTPUT_DIR}/open_ports_summary.txt"
```

5.3 Risoluzione DNS e reverse lookup

■ BASH | DNS resolution, reverse lookup e traceroute con Nmap

```
# Risoluzione DNS durante scan (default: sempre)
nmap --resolve-all 192.168.1.0/24 # risolve tutti gli IP in hostname

# Disabilita risoluzione DNS (piu veloce, nessun traffico DNS)
nmap -n 192.168.1.0/24

# Solo reverse DNS lookup (senza port scan)
nmap -sL 192.168.1.0/24

# Usa DNS server specifico
nmap --dns-servers 8.8.8.8,1.1.1.1 target.example.com

# Traccia route verso il target
nmap --traceroute target.example.com

# Combina traceroute + host discovery
sudo nmap -sn --traceroute 192.168.1.0/24
```

Fonte: nmap.org/book/man-host-discovery.html; RFC 792 — ICMP; RFC 826 — ARP; Fyodor 'Nmap Network Scanning' Cap. 3

6. VERSION DETECTION E OS FINGERPRINTING

Sapere che una porta è aperta è solo il primo passo. Il rilevamento della versione del servizio e del sistema operativo trasforma i dati grezzi in intelligence azionabile per la gestione delle vulnerabilità e l'audit di sicurezza.

6.1 Service Version Detection (-sV)

Nmap interroga ogni porta aperta con una serie di probe specifici, confrontando le risposte con il database nmap-service-probes (oltre 1.200 firme di servizi e versioni):

■ BASH | Service version detection: identifica versioni software sui servizi

```
# Version detection base
nmap -sV 192.168.1.1

# Output di esempio:
# PORT      STATE SERVICE VERSION
# 22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.7 (Ubuntu Linux; protocol 2.0)
# 80/tcp    open  http     Apache httpd 2.4.57 ((Ubuntu))
# 443/tcp   open  ssl/http Apache httpd 2.4.57 ((Ubuntu))
# 3306/tcp  open  mysql    MySQL 8.0.36-0ubuntu0.22.04.1
# 5432/tcp  open  pgsql    PostgreSQL DB 9.6.0 or later

# Intensita version detection (0-9, default 7)
nmap -sV --version-intensity 9 192.168.1.1 # massima (piu lenta)
nmap -sV --version-intensity 0 192.168.1.1 # minima (solo banner grab)

# Versione "light" (piu veloce, meno accurata)
nmap -sV --version-light 192.168.1.1 # intensita 2

# Versione "all" (prova tutti i probe)
nmap -sV --version-all 192.168.1.1 # intensita 9

# Scansione completa: SYN + version + OS
sudo nmap -sS -sV -O 192.168.1.1

# Scansione completa standard di riferimento (la piu usata in audit)
sudo nmap -sV -sC -O -T4 192.168.1.1
# -sV = version detection
# -sC = script default
# -O = OS detection
# -T4 = timing aggressivo
```

6.2 OS Fingerprinting (-O)

Il rilevamento del sistema operativo analizza le risposte TCP/IP del target cercando caratteristiche uniche dello stack di rete: TTL iniziale, dimensione finestra TCP, opzioni TCP, frammentazione IP e altro. Confronta queste caratteristiche con il database nmap-os-db (oltre 5.600 firme OS):

■ BASH | OS fingerprinting: rileva sistema operativo e tipo di dispositivo

```
# OS detection (richiede root)
sudo nmap -O 192.168.1.1

# Output di esempio:
# Device type: general purpose
# Running: Linux 5.X|6.X
# OS CPE: cpe:/o:linux:linux_kernel:5 cpe:/o:linux:linux_kernel:6
# OS details: Linux 5.15 - 6.1
# Network Distance: 1 hop
# OS detection performed. Please report any incorrect results at...

# Aumenta aggressivita OS detection
sudo nmap -O --osscan-guess 192.168.1.1 # indovina anche se incerto
sudo nmap -O --osscan-limit 192.168.1.1 # salta host difficili

# Forza OS detection anche con poche porte aperte
sudo nmap -O --max-os-tries 5 192.168.1.1

# OS + Version detection + script = -A (aggressive)
sudo nmap -A 192.168.1.1
# Equivalente a: -O -sV -sC --traceroute

# Cosa identifica il fingerprinting:
# - Famiglia OS (Linux, Windows, BSD, iOS, Android, Cisco IOS...)
# - Versione kernel/OS
# - Tipo dispositivo (router, switch, webcam, smartphone, printer...)
# - CPE (Common Platform Enumeration) per correlazione CVE

# Limitazioni:
# - Richiede almeno 1 porta aperta E 1 chiusa sul target
# - Dispositivi con firewall possono confondere il fingerprint
# - VPN e NAT possono alterare le caratteristiche TCP/IP
# - Alcuni OS (Windows 11, recenti distro Linux) hanno fingerprint simili
```

6.3 Identificare dispositivi IoT e OT industriali

■ BASH | Rilevamento dispositivi IoT/OT industriali: PLC, Modbus, OPC-UA

```
# Scansione orientata a dispositivi IoT/OT su rete industriale
# ATTENZIONE: su reti OT usare -T2 o -T1 per non sovraccaricare
# dispositivi con risorse limitate (PLC, RTU, sensori)
sudo nmap -sS -sV -O -T2 -p 80,102,443,502,4840,20000,44818,47808 --open 192.168.100.0/24

# Porte OT specifiche da scansionare:
# 102    = Siemens S7 (ISO-TSAP)
# 502    = Modbus/TCP
# 4840   = OPC-UA
# 20000  = DNP3
# 44818  = EtherNet/IP (CIP)
# 47808  = BACnet/IP (UDP)

# Identificare PLC Siemens S7
sudo nmap -sS -p 102 --script s7-info 192.168.100.0/24

# Identificare dispositivi Modbus
sudo nmap -sU -p 502 --script modbus-discover 192.168.100.0/24

# Scansione SNMP per inventory dispositivi
sudo nmap -sU -p 161 --script snmp-info --script-args snmpcommunity=public 192.168.1.0/24
```

Fonte: nmap.org/book/osdetect.html; nmap.org/book/vscan.html; nvd.nist.gov (CVE database); cpe.mitre.org (Common Platform Enumeration)

7. NMAP SCRIPTING ENGINE (NSE)

Il **Nmap Scripting Engine (NSE)** è la caratteristica che ha trasformato Nmap da semplice port scanner a piattaforma di network intelligence. NSE permette di scrivere ed eseguire script Lua che estendono le capacità di Nmap con automazioni avanzate: rilevamento vulnerabilità, raccolta di informazioni, exploiting, brute force e molto altro.

7.1 Categorie di script NSE

Categoria	Descrizione	Esempi script
auth	Autenticazione: bypass, credenziali default, http-headers, http-headers-secure, ftp-anon, ssh-auth-methods	http-headers, http-headers-secure, ftp-anon, ssh-auth-methods
broadcast	Scoperta host tramite broadcast (no target richiesto)	riptide, host-arp-sweep, broadcast-dns-service-discovery
brute	Attacchi brute force su servizi di autenticazione	ssh-brute, ftp-brute, http-brute, smb-brute
default (-sC)	Script sicuri e utili eseguiti con -sC	http-title, ssl-cert, ssh-hostkey, ftp-anon
discovery	Raccolta informazioni: directory, risorse, metadati	dirb, dirbuster, http-robots.txt, snmp-sysdescr
dos	Test Denial of Service (usare solo su sistemi propri)	http-flooder, moris-check, smb-vuln-ms10-054
exploit	Sfruttamento vulnerabilità; specifiche	http-shellshock, smb-vuln-ms17-010 (EternalBlue)
external	Usa API esterne (Shodan, VirusTotal, DNS)	http-virustotal, shodan-api
fuzzer	Invio dati malformati per trovare bug	dns-fuzz, http-form-fuzzer
intrusive	Potenzialmente pericoloso per il target	http-waf-fingerprint, snmp-brute
malware	Rileva backdoor e malware noti	http-malware-host, smtp-strangeport
safe	Sicuro — non danneggia il target	whois-ip, http-headers, ssl-cert
version	Migliora rilevamento versioni servizi	banner, ssh2-enum-algos
vuln	Rileva vulnerabilità; specifiche senza exploit	exploit-ssl-heartbleed, smb-vuln-*

7.2 Eseguire script NSE

■ BASH | Eseguire script NSE: singoli, categorie, wildcard, argomenti

```
# Script di default (categoria "default" – sicuri e informativi)
nmap -sC 192.168.1.1
# equivalente a:
nmap --script=default 192.168.1.1

# Script specifico per nome
nmap --script http-title 192.168.1.1
nmap --script ssl-cert 192.168.1.1

# Più script separati da virgola
nmap --script http-title,http-headers,http-methods 192.168.1.1

# Tutti gli script di una categoria
nmap --script "vuln" 192.168.1.1 # tutti gli script vuln
nmap --script "safe and discovery" 192.168.1.1 # safe AND discovery
nmap --script "not intrusive" 192.168.1.1 # escludi intrusive

# Script con wildcard
nmap --script "http-*" 192.168.1.1 # tutti gli script http-*
nmap --script "smb-vuln-*" 192.168.1.1 # tutti smb-vuln-*

# Passare argomenti agli script
nmap --script http-brute --script-args userdb=/path/users.txt,passdb=/path/pass.txt 192.168.1.1

nmap --script snmp-brute --script-args brute.mode=user 192.168.1.1

# Argomenti multipli (sintassi con virgola)
nmap --script dns-brute --script-args dns-brute.domain=example.com,dns-brute.threads=10 192.168.1.1

# Debug degli script
nmap --script http-title --script-trace 192.168.1.1 # mostra traffico
nmap -d5 --script http-title 192.168.1.1 # massimo debug

# Dove sono gli script?
ls /usr/share/nmap/scripts/ # Linux
ls /usr/local/share/nmap/scripts/ # macOS (Homebrew)
# C:\Program Files (x86)\Nmap\scripts\ # Windows
```

7.3 Script NSE essenziali per security audit

■ BASH | Script NSE essenziali: SSL, SMB, HTTP, vulnerabilità, database

[illegible]

```
nmap -p 27017 --script mongodb-info 192.168.1.1
```

```
# Redis: accesso non autenticato
```

```
nmap -p 6379 --script redis-info 192.168.1.1
```

7.4 Scrivere script NSE personalizzati in Lua

■ LUA | custom_banner.nse — script NSE personalizzato in Lua

```
-- custom_banner.nse
-- Script NSE personalizzato: cattura banner da porta specificata
-- Posizionare in /usr/share/nmap/scripts/
-- Aggiornare database: nmap --script-updatedb

local nmap = require "nmap"
local shortport = require "shortport"
local stdnse = require "stdnse"

-- Descrizione dello script (mostrata con --script-help)
description = [[
Cattura il banner di benvenuto da qualsiasi servizio TCP.
Utile per rilevare software e versioni non standard.
]]

-- Metadati
author = "Mario Rossi"
license = "Same as Nmap"
categories = {"discovery", "safe"}

-- Regola di esecuzione: quando applicare questo script
-- shortport.port_or_service attiva su porta specifica o nome servizio
portrule = shortport.port_or_service({21, 22, 25, 80, 110, 143, 443, 8080},
    {"ftp", "ssh", "smtp", "http", "pop3", "imap", "https"})

-- Funzione principale
action = function(host, port)
    -- Apri connessione TCP
    local socket = nmap.new_socket()
    local status, err = socket:connect(host, port)

    if not status then
        return stdnse.format_output(false, "Connessione fallita: " .. err)
    end

    -- Ricevi banner (timeout 5 secondi)
    socket:set_timeout(5000)
    local status2, data = socket:receive_lines(1)
    socket:close()

    if status2 and data then
        -- Pulisci il banner (rimuovi caratteri di controllo)
        data = data:gsub("[\x00-\x1f]", "")
        return "Banner: " .. data
    else
        return "Nessun banner ricevuto"
    end
end

-- Uso: nmap --script custom_banner -p 21,22,25,80 192.168.1.1
```

■ LUA | port_checker.nse — verifica SSL/TLS con rilevamento problemi

```
-- port_checker.nse
-- Script piu avanzato: verifica configurazione SSL e riporta problemi
local nmap = require "nmap"
local shortport = require "shortport"
local sslcert = require "sslcert"
local stdnse = require "stdnse"
local datetime = require "datetime"

description = [[
Verifica la configurazione SSL/TLS e segnala problemi di sicurezza:
certificato scaduto, algoritmi deboli, Self-signed certificate.
]]

author = "Security Team"
license = "Same as Nmap"
categories = {"safe", "vuln"}

portrule = shortport.ssl

action = function(host, port)
    -- Recupera il certificato SSL
    local status, cert = sslcert.getCertificate(host, port)

    if not status then
        return "Impossibile recuperare certificato SSL"
    end

    local output = stdnse.output_table()
    local issues = {}

    -- Controlla scadenza
    local not_after = cert.notAfter()
    local now = os.time()
    if not_after < now then
        table.insert(issues, "CRITICO: Certificato SCADUTO il " ..
            datetime.format_timestamp(not_after))
    elseif not_after < (now + 30*24*60*60) then
        table.insert(issues, "ATTENZIONE: Certificato scade entro 30 giorni")
    end

    -- Controlla self-signed
    if cert.issuer == cert.subject then
        table.insert(issues, "Certificato self-signed (non fidato)")
    end

    output.subject = cert.subject.commonName or "N/A"
    output.issuer = cert.issuer.commonName or "N/A"
    output.expires = datetime.format_timestamp(not_after)

    if #issues > 0 then
        output.issues = issues
    else
        output.status = "OK - Nessun problema rilevato"
    end

    return output
end
```

end

-- Uso: nmap -p 443 --script port_checker 192.168.1.1

Fonte: nmap.org/book/nse.html; nmap.org/nsedoc/; lua.org/manual/5.4/ (Lua reference); nmap.org/book/nse-api.html (NSE API reference)

8. EVASIONE FIREWALL E IDS

Nmap include numerose tecniche per aggirare firewall, packet filter e sistemi di rilevamento delle intrusioni (IDS/IPS). Queste tecniche sono fondamentali per i penetration tester che devono simulare attacchi reali, e per i difensori che vogliono verificare l'efficacia dei propri controlli.

■ **SOLO USO AUTORIZZATO** Le tecniche di evasione descritte in questo capitolo devono essere utilizzate SOLO su infrastrutture proprie o con autorizzazione scritta. Il loro uso non autorizzato è un reato penale.

8.1 Frammentazione dei pacchetti

■ BASH | Frammentazione pacchetti: elude ispezione firewall stateless

```
# Frammentazione pacchetti (elude firewall che non riassemblano)
sudo nmap -f 192.168.1.1      # frammenta in 8 byte
sudo nmap -ff 192.168.1.1    # frammenta in 16 byte
sudo nmap --mtu 16 192.168.1.1 # MTU personalizzato (multiplo di 8)
sudo nmap --mtu 24 192.168.1.1

# Come funziona:
# Nmap divide i pacchetti TCP/IP in frammenti piu piccoli
# Alcuni firewall/IDS non riassemblano i frammenti prima dell'ispezione
# I frammenti passano singolarmente ma il target li riassembra correttamente

# Verifica frammentazione con tcpdump (su un altro terminale):
# sudo tcpdump -i eth0 host 192.168.1.1 -v
```

8.2 Decoy scan: mascherare l'origine

■ BASH | Decoy scan: mescola IP fasulli per nascondere lo scanner

```
# Decoy scan: usa IP fasulli (decoy) mescolati al tuo reale
# Il target vede traffico da piu IP: difficile isolare il reale scanner
sudo nmap -D 10.0.0.1,10.0.0.2,10.0.0.3 192.168.1.1

# ME indica la tua posizione nella lista decoy
sudo nmap -D 10.0.0.1,10.0.0.2,ME,10.0.0.3 192.168.1.1

# Usa IP casuali come decoy
sudo nmap -D RND:10 192.168.1.1 # 10 decoy casuali

# ATTENZIONE: i decoy devono essere host attivi!
# Se i decoy sono down -> piu facile isolare lo scanner reale
# Verifica prima che gli IP decoy siano attivi:
sudo nmap -sn 10.0.0.1-5 # controlla quali sono up

# Decoy + timing lento per massima evasione
sudo nmap -D RND:5 -T1 --randomize-hosts 192.168.1.0/24
```

8.3 IP Spoofing e Source Port manipulation

■ BASH | IP spoofing e source port manipulation

```
# Spoofing IP sorgente (attenzione: non riceverai le risposte!)
sudo nmap -S 10.0.0.5 -e eth0 192.168.1.1
# -S = IP sorgente falsificato
# -e = interfaccia di rete da usare

# Porta sorgente fissa (elude firewall che permettono traffico da porta 53/80)
# Alcuni firewall permettono traffico in ingresso da porta 53 (DNS) o 80 (HTTP)
sudo nmap --source-port 53 192.168.1.1 # simula traffico DNS
sudo nmap -g 53 192.168.1.1           # equivalente con -g
sudo nmap --source-port 80 192.168.1.1 # simula traffico HTTP

# Perché funziona:
# Firewall con regola "permetti TCP sorgente=53" -> interpreta come DNS legittimo
# Il target vede la porta 53 come sorgente e risponde normalmente
```

8.4 Randomizzazione e delay

■ BASH | Randomizzazione, delay, MAC spoofing e tecniche avanzate di evasione

```
# Ordine casuale degli host (default: già casuale)
nmap --randomize-hosts 192.168.1.0/24

# Delay tra probe (elude rate-based IDS)
nmap --scan-delay 1s 192.168.1.1 # 1 secondo tra probe
nmap --scan-delay 500ms 192.168.1.1 # 500 millisecondi
nmap --max-scan-delay 5s 192.168.1.1 # delay adattivo max 5 sec

# Numero max di retransmission (riduce rumore)
nmap --max-retries 1 192.168.1.1

# Disabilita ping (elude blocco ICMP)
nmap -Pn --max-retries 1 --scan-delay 1s 192.168.1.1

# Append random data ai pacchetti (elude signature IDS)
sudo nmap --data-length 25 192.168.1.1 # aggiunge 25 byte casuali
sudo nmap --data-length 50 192.168.1.1

# Spoof MAC address (solo su LAN)
sudo nmap --spoof-mac 0 192.168.1.1 # MAC casuale
sudo nmap --spoof-mac "Apple" 192.168.1.1 # MAC Apple casuale
sudo nmap --spoof-mac 00:11:22:33:44:55 192.168.1.1 # MAC specifico

# Impostare TTL personalizzato
sudo nmap --ttl 64 192.168.1.1 # TTL tipico Linux
sudo nmap --ttl 128 192.168.1.1 # TTL tipico Windows

# Badsum: pacchetti con checksum errato
# Host reali scartano i pacchetti -> firewall/IDS potrebbero passarli
sudo nmap --badsum 192.168.1.1
```

8.5 Scanning attraverso proxy e Tor

■ BASH | Scansione via proxy SOCKS5 e Tor con proxychains

```
# Scansione tramite proxy SOCKS5 (es: Tor)
# ATTENZIONE: SYN scan non funziona via proxy (raw socket necessari)
# Solo TCP Connect scan (-sT) funziona tramite proxy

# Installa proxychains (Linux)
sudo apt install proxychains4

# Configura /etc/proxychains4.conf:
# [ProxyList]
# socks5 127.0.0.1 9050 # Tor SOCKS5

# Avvia Tor
sudo systemctl start tor

# Scansione via Tor (solo -sT, niente raw socket)
proxychains nmap -sT -Pn -n --top-ports 100 192.168.1.1

# Via proxy HTTP esplicito con Ncat
ncat --proxy 127.0.0.1:8080 --proxy-type http 192.168.1.1 80

# Limitazioni proxy:
# - Molto piu lento (latenza Tor = centinaia di ms)
# - Solo TCP Connect (no SYN, no UDP, no ICMP)
# - Alcune scan NSE non funzionano
# - Exit node Tor potrebbe loggare il traffico
```

Tecnica evasione	Contro cosa funziona	Limiti
-f (frammentazione)	Firewall stateless senza riassetblaggio	IDS moderni riassemblano i frammenti
-D (decoy)	Log analisi — difficile isolare scanner	Target vede tutti gli IP decoy nel log
--source-port 53/80	Firewall con regole basate su porta sorgente	Firewall stateful ignorano porta sorgente
-T0/-T1 (timing lento)	IDS con soglie di rate	Scansione dura ore/giorni
--scan-delay	IDS con correlazione temporale	Scansione molto lenta
--data-length	IDS con signature su dimensione payload	IDS avanzati ignorano il payload
--spoof-mac	Limitati a LAN locale	Non funziona su internet
proxychains + Tor	Tracciabilità IP	Solo TCP Connect, lentissimo
-sl (idle scan)	Tutto — 100% anonimo	Richiede zombie con IP ID prevedibile

Fonte: nmap.org/book/man-bypass-firewalls-ids.html; [proxychains4 \(github.com/haad/proxychains\)](https://github.com/haad/proxychains); [Tor Project \(torproject.org\)](https://torproject.org)

9. AUTOMAZIONE CON PYTHON

Python è il linguaggio ideale per automatizzare scansioni Nmap, parsare i risultati e integrarli in pipeline di sicurezza. La libreria python-nmap fornisce un wrapper Pythonic per l'esecuzione e il parsing dell'output XML di Nmap.

9.1 Installazione python-nmap

■ BASH | Installazione python-nmap

```
pip install python-nmap
```

```
# Verifica
```

```
python3 -c "import nmap; print(nmap.__version__)"
```

```
# Dipendenza: Nmap deve essere installato sul sistema
```

```
# python-nmap e' solo un wrapper che chiama il binario nmap
```

9.2 Utilizzo base di python-nmap

■ PYTHON | nmap_base.py — utilizzo base python-nmap: scan, iter host e porte

[illegible]

```
xml_output = nm.get_nmap_last_output() # stringa XML completa
```

9.3 Scanner di rete avanzato con reportistica

■ PYTHON | network_scanner.py — scanner completo con output JSON e CSV

```
#!/usr/bin/env python3
"""
network_scanner.py
Scanner di rete completo con reportistica CSV e JSON.
Dipendenze: pip install python-nmap
Uso: sudo python3 network_scanner.py --target 192.168.1.0/24
"""

import nmap
import json
import csv
import argparse
import sys
from datetime import datetime
from pathlib import Path

def scan_network(target: str, ports: str = "1-1024",
                 arguments: str = "-sS -sV -O -T4") -> dict:
    """Esegue scansione completa e restituisce risultati strutturati."""
    nm = nmap.PortScanner()
    print(f"[*] Avvio scansione: {target}")
    print(f"[*] Porte: {ports} | Argomenti: {arguments}")
    print(f"[*] Inizio: {datetime.now():%Y-%m-%d %H:%M:%S}")

    try:
        nm.scan(hosts=target, ports=ports, arguments=arguments, sudo=True)
    except nmap.PortScannerError as e:
        print(f"[!] Errore: {e}", file=sys.stderr)
        sys.exit(1)

    results = {
        "scan_info": {
            "target": target,
            "ports": ports,
            "arguments": arguments,
            "command": nm.command_line(),
            "timestamp": datetime.now().isoformat(),
        },
        "hosts": []
    }

    for host in nm.all_hosts():
        host_data = {
            "ip": host,
            "hostname": nm[host].hostname(),
            "state": nm[host].state(),
            "os": [],
            "ports": []
        }

        # OS detection
        if 'osmatch' in nm[host]:
            for osmatch in nm[host]['osmatch'][:3]:
                host_data["os"].append({
                    "name": osmatch.get('name', ''),
                    "accuracy": osmatch.get('accuracy', ''),

```

```

    })

    # Porte
    for proto in nm[host].all_protocols():
        for port in sorted(nm[host][proto].keys()):
            p = nm[host][proto][port]
            port_data = {
                "port": port,
                "protocol": proto,
                "state": p['state'],
                "service": p.get('name', ''),
                "product": p.get('product', ''),
                "version": p.get('version', ''),
                "extrainfo": p.get('extrainfo', ''),
                "cpe": p.get('cpe', ''),
            }
            if p['state'] == 'open':
                host_data["ports"].append(port_data)

        results["hosts"].append(host_data)

    return results

def save_json(results: dict, filepath: str):
    with open(filepath, 'w', encoding='utf-8') as f:
        json.dump(results, f, indent=2, ensure_ascii=False)
    print(f"[✓] JSON: {filepath}")

def save_csv(results: dict, filepath: str):
    with open(filepath, 'w', newline='', encoding='utf-8') as f:
        writer = csv.writer(f)
        writer.writerow(['IP', 'Hostname', 'OS', 'Porta', 'Proto', 'Stato', 'Servizio', 'Prodotto', 'Versione'])
        for host in results['hosts']:
            os_name = host['os'][0]['name'] if host['os'] else 'N/A'
            if host['ports']:
                for p in host['ports']:
                    writer.writerow([
                        host['ip'], host['hostname'], os_name,
                        p['port'], p['protocol'], p['state'],
                        p['service'], p['product'], p['version']
                    ])
            else:
                writer.writerow([host['ip'], host['hostname'], os_name,
                                '-', '-', host['state'], '-', '-', '-'])
    print(f"[✓] CSV: {filepath}")

def print_summary(results: dict):
    hosts = results['hosts']
    up = [h for h in hosts if h['state'] == 'up']
    print(f"{'='*60}")
    print(f" RIEPILOGO SCANSIONE")
    print(f" Target: {results['scan_info']['target']}")
    print(f" Host attivi: {len(up)} / {len(hosts)}")
    print(f"{'='*60}")
    for host in up:

```

```

        os_str = host['os'][0]['name'][:40] if host['os'] else 'OS sconosciuto'
        print(f"
{host['ip']:16s} {host['hostname'] or '':20s}")
        print(f"  OS: {os_str}")
        if host['ports']:
            print(f"  Porte aperte ({len(host['ports'])}):")
            for p in host['ports'][:10]:
                svc = f"{p['product']} {p['version']}".strip() or p['service']
                print(f"    {p['port']:5d}/{p['protocol']:3s} {svc[:50]}")
            if len(host['ports']) > 10:
                print(f"    ... e altre {len(host['ports'])-10} porte")
        print(f"
{' '*60}
")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Network Scanner con python-nmap")
    parser.add_argument("--target", required=True, help="Target: IP, range o CIDR")
    parser.add_argument("--ports", default="1-1024", help="Range porte (default: 1-1024)")
    parser.add_argument("--args", default="-sS -sV -O -T4", help="Argomenti nmap extra")
    parser.add_argument("--output", default="./scan_results", help="Directory output")
    args = parser.parse_args()

    outdir = Path(args.output)
    outdir.mkdir(exist_ok=True)
    timestamp = datetime.now().strftime("%Y%m%d_%H%M")

    results = scan_network(args.target, args.ports, args.args)
    print_summary(results)
    save_json(results, outdir / f"scan_{timestamp}.json")
    save_csv(results, outdir / f"scan_{timestamp}.csv")

```

9.4 Vulnerability scanner automatizzato

■ PYTHON | vuln_scanner.py — scanner vulnerabilità automatizzato con python-nmap

```
#!/usr/bin/env python3
"""
vuln_scanner.py
Scanner vulnerabilita automatico con Nmap NSE.
Controlla le vulnerabilita piu critiche in modo sistematico.
Dipendenze: pip install python-nmap
Uso: sudo python3 vuln_scanner.py --target 192.168.1.1
"""

import nmap
import json
import sys
import argparse
from datetime import datetime

# Definizione controlli di sicurezza da eseguire
SECURITY_CHECKS = [
    {
        "name": "EternalBlue (CVE-2017-0144)",
        "port": 445,
        "protocol": "tcp",
        "script": "smb-vuln-ms17-010",
        "severity": "CRITICA",
        "description": "RCE su SMB – sfruttato da WannaCry e NotPetya"
    },
    {
        "name": "Heartbleed (CVE-2014-0160)",
        "port": 443,
        "protocol": "tcp",
        "script": "ssl-heartbleed",
        "severity": "CRITICA",
        "description": "Memory disclosure su OpenSSL – espone chiavi private"
    },
    {
        "name": "BlueKeep (CVE-2019-0708)",
        "port": 3389,
        "protocol": "tcp",
        "script": "rdp-vuln-ms12-020",
        "severity": "CRITICA",
        "description": "RCE pre-autenticazione su RDP Windows"
    },
    {
        "name": "FTP Anonimo",
        "port": 21,
        "protocol": "tcp",
        "script": "ftp-anon",
        "severity": "ALTA",
        "description": "Accesso FTP senza autenticazione"
    },
    {
        "name": "SNMP Community String 'public'",
        "port": 161,
        "protocol": "udp",
        "script": "snmp-brute",
        "severity": "ALTA",
        "description": "Community string SNMP predefinita esposta"
    }
]
```

```

    },
    {
        "name":      "SSL Self-Signed / Scaduto",
        "port":      443,
        "protocol":  "tcp",
        "script":     "ssl-cert",
        "severity":   "MEDIA",
        "description": "Certificato SSL non valido o scaduto"
    },
    {
        "name":      "SSH Algoritmi Deboli",
        "port":      22,
        "protocol":  "tcp",
        "script":     "ssh2-enum-algos",
        "severity":   "MEDIA",
        "description": "SSH supporta algoritmi crittografici obsoleti"
    },
    {
        "name":      "MySQL Senza Password",
        "port":      3306,
        "protocol":  "tcp",
        "script":     "mysql-empty-password",
        "severity":   "CRITICA",
        "description": "Database MySQL accessibile senza password"
    },
]

def run_check(nm: nmap.PortScanner, target: str, check: dict) -> dict:
    """Esegue un singolo controllo di sicurezza."""
    port = str(check['port'])
    proto = check['protocol']
    script = check['script']

    # Costruisce argomenti nmap
    scan_args = f"-sV --script {script} -T4"
    if proto == 'udp':
        scan_args = f"-sU --script {script} -T4"

    try:
        nm.scan(hosts=target, ports=port, arguments=scan_args, sudo=True)
    except Exception as e:
        return {"status": "ERROR", "error": str(e)}

    # Analizza risultato
    result = {"status": "NOT_CHECKED", "details": ""}
    if target in nm.all_hosts():
        host = nm[target]
        port_int = int(port)
        if proto in host and port_int in host[proto]:
            port_state = host[proto][port_int]['state']
            if port_state == 'open':
                # Cerca output script
                scripts = host[proto][port_int].get('script', {})
                if script in scripts:
                    output = scripts[script]
                    # Cerca pattern di vulnerabilita

```

```

        if 'VULNERABLE' in output.upper() or 'State: VULNERABLE' in output:
            result = {"status": "VULNERABLE", "details": output[:500]}
        elif 'Anonymous FTP login allowed' in output:
            result = {"status": "VULNERABLE", "details": output[:200]}
        else:
            result = {"status": "NOT_VULNERABLE", "details": output[:200]}
    else:
        result = {"status": "SCRIPT_NO_OUTPUT", "details": "Porta aperta ma script senza o"}
    else:
        result = {"status": "PORT_CLOSED", "details": f"Porta {port} e' {port_state}"}
    else:
        result = {"status": "PORT_NOT_FOUND", "details": ""}
return result

def vulnerability_report(target: str, checks: list) -> dict:
    nm = nmap.PortScanner()
    report = {
        "target": target,
        "timestamp": datetime.now().isoformat(),
        "summary": {"critica": 0, "alta": 0, "media": 0, "bassa": 0},
        "findings": []
    }

    print(f"
{'='*60}")
    print(f"  VULNERABILITY SCAN: {target}")
    print(f"  {len(checks)} controlli da eseguire")
    print(f"{'='*60}")
    ")

    for i, check in enumerate(checks, 1):
        print(f"[{i:2d}/{len(checks)}] {check['name']}", end=" ", flush=True)
        result = run_check(nm, target, check)
        status = result['status']

        finding = {**check, **result}
        report['findings'].append(finding)

        if status == 'VULNERABLE':
            sev = check['severity'].lower()
            report['summary'][sev] = report['summary'].get(sev, 0) + 1
            print(f"[!] VULNERABILE ({check['severity']})")
        elif status == 'NOT_VULNERABLE':
            print("[OK] Non vulnerabile")
        elif status == 'PORT_CLOSED':
            print("[--] Porta chiusa")
        else:
            print(f"[?] {status}")

    # Stampa riepilogo
    print(f"
{'='*60}")
    print(f"  RIEPILOGO VULNERABILITA' TROVATE")
    print(f"  Critiche: {report['summary'].get('critica', 0)}")
    print(f"  Alte:     {report['summary'].get('alta', 0)}")
    print(f"  Medie:    {report['summary'].get('media', 0)}")

```

```

        print(f"{'='*60}
    ")

    vulnerable = [f for f in report['findings'] if f['status'] == 'VULNERABLE']
    if vulnerable:
        print(" DETTAGLIO VULNERABILITA' TROVATE:")
        for v in vulnerable:
            print(f"
[{'severity'}] {'name'}")
            print(f" Descrizione: {'description'}")
            print(f" Porta: {'port'}/{v['protocol']}")
            if v.get('details'):
                print(f" Output script:
{'details'}[:300}")

        return report

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--target", required=True, help="IP o hostname target")
    parser.add_argument("--output", default="vuln_report.json")
    args = parser.parse_args()

    report = vulnerability_report(args.target, SECURITY_CHECKS)
    with open(args.output, 'w') as f:
        json.dump(report, f, indent=2, ensure_ascii=False)
    print(f"
[✓] Report salvato: {args.output}")

```

Fonte: [python-nmap: xael.org/pages/python-nmap.html](https://xael.org/pages/python-nmap.html); github.com/nmmapper/python3-nmap (alternativa); docs.python.org/3/library/subprocess.html

10. INTEGRAZIONE CON METASPLOIT E WIRESHARK

Nmap si integra nativamente con Metasploit Framework e complementa perfettamente Wireshark nel workflow di un penetration test. Questa sezione mostra come sfruttare queste integrazioni.

10.1 Nmap + Metasploit Framework

■ BASH | Integrazione Nmap + Metasploit: import XML, db_nmap, workflow exploit

[illegible]

■ **SOLO USO AUTORIZZATO** L'uso di Metasploit per exploit reali richiede autorizzazione esplicita scritta. L'esecuzione di exploit non autorizzati costituisce reato grave in tutte le giurisdizioni.

10.2 Nmap + Wireshark: analisi del traffico di scansione

■ BASH | Cattura e analisi traffico Nmap con Wireshark e tshark

[illegible]

10.3 Parsing output XML con Python

■ PYTHON | parse_nmap_xml.py — parser XML completo con report HTML e JSON

```
#!/usr/bin/env python3
"""
parse_nmap_xml.py
Parser completo per output XML di Nmap.
Genera report HTML, CSV e statistiche.
"""

import xml.etree.ElementTree as ET
import json
import csv
from pathlib import Path
from datetime import datetime

def parse_nmap_xml(xml_file: str) -> dict:
    """Parsa un file XML di Nmap e restituisce struttura dati Python."""
    tree = ET.parse(xml_file)
    root = tree.getroot()

    # Metadati scansione
    scan_info = {
        "command": root.get('args', ''),
        "version": root.get('version', ''),
        "start": root.get('startstr', ''),
        "elapsed": root.find('runstats/finished').get('elapsed', '') if root.find('runstats/finished')
        "hosts_up": root.find('runstats/hosts').get('up', '0') if root.find('runstats/hosts') is not None
        "hosts_down": root.find('runstats/hosts').get('down', '0') if root.find('runstats/hosts') is not None
    }

    hosts = []
    for host_elem in root.findall('host'):
        # Stato host
        status = host_elem.find('status')
        state = status.get('state', '') if status is not None else ''
        if state != 'up':
            continue

        # Indirizzi IP/MAC
        addresses = {}
        for addr in host_elem.findall('address'):
            addr_type = addr.get('addrtype', '')
            addresses[addr_type] = addr.get('addr', '')

        # Hostname
        hostnames = []
        for hn in host_elem.findall('hostnames/hostname'):
            hostnames.append(hn.get('name', ''))

        # OS detection
        os_matches = []
        for osmatch in host_elem.findall('os/osmatch'):
            os_matches.append({
                'name': osmatch.get('name', ''),
                'accuracy': int(osmatch.get('accuracy', 0)),
                'line': osmatch.get('line', ''),
            })
        os_matches.sort(key=lambda x: x['accuracy'], reverse=True)
```



```

# Porte
ports = []
for port_elem in host_elem.findall('ports/port'):
    state_elem = port_elem.find('state')
    if state_elem is None:
        continue

    service_elem = port_elem.find('service')
    service = {}
    if service_elem is not None:
        service = {
            'name':      service_elem.get('name', ''),
            'product':   service_elem.get('product', ''),
            'version':   service_elem.get('version', ''),
            'extrainfo': service_elem.get('extrainfo', ''),
            'cpe':       service_elem.get('cpe', ''),
            'tunnel':    service_elem.get('tunnel', ''),
        }

# Script output
scripts = {}
for script_elem in port_elem.findall('script'):
    scripts[script_elem.get('id', '')] = script_elem.get('output', '')

port_data = {
    'port':      int(port_elem.get('portid', 0)),
    'protocol':  port_elem.get('protocol', ''),
    'state':     state_elem.get('state', ''),
    'reason':    state_elem.get('reason', ''),
    'service':   service,
    'scripts':   scripts,
}
ports.append(port_data)

open_ports = [p for p in ports if p['state'] == 'open']

hosts.append({
    'ip':        addresses.get('ipv4', ''),
    'mac':        addresses.get('mac', ''),
    'hostnames':  hostnames,
    'os':         os_matches[:3],
    'ports':     open_ports,
    'all_ports': ports,
})

return {"scan_info": scan_info, "hosts": hosts}

def export_html(data: dict, output_file: str):
    """Genera report HTML con tabelle colorate."""
    html = f"""<!DOCTYPE html>
<html lang="it">
<head>
<meta charset="UTF-8">
<title>Nmap Report - {data['scan_info']['start']}</title>
<style>
    body {{ font-family: 'Segoe UI', sans-serif; background:#f5f5f5; color:#333; }}
    """

```

```

h1 {{ background:#0a0f1e; color:#00b4d8; padding:20px; margin:0; }}
.info {{ background:#e0f7fa; padding:10px 20px; border-left:4px solid #00b4d8; }}
table {{ border-collapse:collapse; width:100%; margin:20px 0; }}
th {{ background:#1a3a5c; color:white; padding:10px; text-align:left; }}
td {{ padding:8px 10px; border-bottom:1px solid #ddd; }}
tr:nth-child(even) {{ background:#f0f4f8; }}
.open {{ color:#06d6a0; font-weight:bold; }}
.closed {{ color:#e63946; }}
.host-card {{ background:white; margin:20px; padding:20px;
              border-radius:8px; box-shadow:0 2px 8px rgba(0,0,0,0.1); }}
.badge {{ display:inline-block; padding:2px 8px; border-radius:4px;
          background:#0a0f1e; color:#00b4d8; font-size:0.85em; }}

</style>
</head>
<body>
<h1>&#128270; Nmap Security Report</h1>
<div class="info">
  <strong>Comando:</strong> {data['scan_info']['command']}<br>
  <strong>Data:</strong> {data['scan_info']['start']}<br>
  <strong>Durata:</strong> {data['scan_info']['elapsed']}s |
  <strong>Host UP:</strong> {data['scan_info']['hosts_up']} |
  <strong>Host DOWN:</strong> {data['scan_info']['hosts_down']}
</div>
"""
    for host in data['hosts']:
        ip = host['ip']
        hostname = ', '.join(host['hostnames']) or 'N/A'
        os_str = host['os'][0]['name'] if host['os'] else 'N/A'

        html += f"""
<div class="host-card">
  <h2>&#128421; {ip} <span class="badge">{hostname}</span></h2>
  <p><strong>Sistema Operativo:</strong> {os_str}</p>
  <table>
    <tr><th>Porta</th><th>Protocollo</th><th>Stato</th>
      <th>Servizio</th><th>Prodotto / Versione</th></tr>
    """
        for p in host['ports']:
            svc = p['service']
            product_ver = f"{svc.get('product','')} {svc.get('version','')}".strip()
            state_class = 'open' if p['state'] == 'open' else 'closed'
            html += f"""      <tr>
        <td><strong>{p['port']}</strong></td>
        <td>{p['protocol']}</td>
        <td class="{state_class}">{p['state']}</td>
        <td>{svc.get('name','')}</td>
        <td>{product_ver}</td>
      </tr>
    """
        html += "    </table>
  </div>
  """

    html += f"""<footer style="text-align:center;padding:20px;color:#888;font-size:0.85em">
Generato il {datetime.now():%Y-%m-%d %H:%M:%S}
</footer>

```

```
</body></html>""

    with open(output_file, 'w', encoding='utf-8') as f:
        f.write(html)
    print(f"[✓] HTML: {output_file}")

if __name__ == "__main__":
    import sys
    xml_file = sys.argv[1] if len(sys.argv) > 1 else "scan.xml"
    data = parse_nmap_xml(xml_file)
    export_html(data, "nmap_report.html")
    with open("nmap_report.json", 'w') as f:
        json.dump(data, f, indent=2)
    print(f"[✓] JSON: nmap_report.json")
    print(f"[✓] Host analizzati: {len(data['hosts'])}")
```

Fonte: nmap.org/book/output-formats-xml-output.html; metasploit.com/; docs.metasploit.com/; wireshark.org/docs/wsug_html_chunked/; github.com/rapid7/metasploit-framework

11. DIFESA: RILEVARE E BLOCCARE SCANSIONI

Comprendere come funziona Nmap è fondamentale anche per i difensori. Questa sezione mostra come rilevare scansioni Nmap in corso, bloccarle con regole firewall e minimizzare la superficie di attacco.

11.1 Rilevare scansioni con Snort e Suricata

■ BASH | Regole Snort/Suricata per rilevare scansioni Nmap

```
# Rileva SYN scan (molti SYN senza completamento handshake)
alert tcp any any -> $HOME_NET any (
    msg:"NMAP SYN Scan Detected";
    flags:S;
    threshold:type both, track by_src, count 20, seconds 3;
    classtype:network-scan;
    sid:1000001; rev:1;
)

# Rileva NULL scan (pacchetti TCP senza flag)
alert tcp any any -> $HOME_NET any (
    msg:"NMAP NULL Scan";
    flags:0;
    classtype:network-scan;
    sid:1000002; rev:1;
)

# Rileva XMAS scan (FIN+PSH+URG)
alert tcp any any -> $HOME_NET any (
    msg:"NMAP XMAS Scan";
    flags:FPU;
    classtype:network-scan;
    sid:1000003; rev:1;
)

# Rileva FIN scan
alert tcp any any -> $HOME_NET any (
    msg:"NMAP FIN Scan";
    flags:F;
    classtype:network-scan;
    sid:1000004; rev:1;
)

# Rileva port scan orizzontale (stesso IP, molte porte diverse)
alert tcp any any -> $HOME_NET any (
    msg:"NMAP Port Scan - Horizontal";
    threshold:type both, track by_src, count 50, seconds 10;
    classtype:network-scan;
    sid:1000005; rev:1;
)

# Rileva NMAP OS detection (pacchetto con TCP options specifiche)
alert tcp any any -> $HOME_NET any (
    msg:"NMAP OS Detection Attempt";
    flags:S;
    tcp.opts;content:"|02 04 05 b4 04 02 08 0a|";
    classtype:network-scan;
    sid:1000006; rev:1;
)
```

11.2 Bloccare scansioni con iptables e nftables

■ BASH | iptables, nftables e fail2ban: blocco automatico scansioni

[illegible]

```
cat > /etc/fail2ban/jail.d/nmap.conf << 'EOF'
[nmap-scan]
enabled = true
filter = nmap-scan
logpath = /var/log/kern.log
maxretry = 3
bantime = 3600
findtime = 60
EOF
systemctl restart fail2ban
```

11.3 Hardening: ridurre la superficie di attacco

Misura	Implementazione	Impatto su Nmap
Chiudi porte non necessarie	systemctl disable servizio; ufw deny port	Riduce superficie — meno informazioni da rilevamento versioni
Firewall con default-deny	iptables -P INPUT DROP + whitelist	Porte sembrano 'filtered' invece di 'open' — Nmap non rileva servizi
Blocca ICMP echo	iptables -A INPUT -p icmp -j DROP	Host sembrano 'down' a ping sweep — Nmap richiede -Pn
Cambia porte predefinite	SSH su 2222, HTTP su 8080	Top-1000 scan non rileva — richiede -p- (full scan)
Port knocking	knockd: sequenza porte per sbloccare servizi	Sembra SSH appare closed finche non si batte la sequenza
Honeypot	HoneyD, OpenCanary: simula servizi fasulli	Confonde scanner — genera alert su tentativi di connessione
Rate limiting connessioni	iptables -m limit --limit 5/s	Rallenta drasticamente scansioni aggressive
Disabilita TCP timestamp	echo 0 > /proc/sys/net/ipv4/tcp_timestamps	Ostacola OS fingerprinting (timestamp è una firma OS)
Randomizza TTL	Vari metodi kernel/iptables	Ostacola rilevamento OS basato su TTL

■ BASH | hardening_script.sh — misure di hardening anti-scansione su Linux

```
#!/bin/bash
# hardening_script.sh – Script di hardening anti-scansione
# ATTENZIONE: testare in ambienti non produzione prima!

echo "[*] Applicazione misure anti-scansione..."

# 1. Disabilita risposte ICMP (nasconde host)
echo 1 > /proc/sys/net/ipv4/icmp_echo_ignore_all
# Rendi permanente:
echo "net.ipv4.icmp_echo_ignore_all = 1" >> /etc/sysctl.conf

# 2. Disabilita TCP timestamp (ostacola OS fingerprinting)
echo 0 > /proc/sys/net/ipv4/tcp_timestamps
echo "net.ipv4.tcp_timestamps = 0" >> /etc/sysctl.conf

# 3. Randomizza ID IP (ostacola idle scan)
# (già default su Linux moderno)

# 4. Aumenta backlog SYN (mitiga SYN flood)
echo 2048 > /proc/sys/net/ipv4/tcp_max_syn_backlog
echo "net.ipv4.tcp_max_syn_backlog = 2048" >> /etc/sysctl.conf

# 5. Abilita SYN cookies (protezione SYN flood)
echo 1 > /proc/sys/net/ipv4/tcp_syncookies
echo "net.ipv4.tcp_syncookies = 1" >> /etc/sysctl.conf

# 6. iptables: blocca scansioni comuni
iptables -A INPUT -p tcp --tcp-flags ALL NONE -j DROP      # NULL scan
iptables -A INPUT -p tcp --tcp-flags ALL ALL -j DROP      # XMAS scan
iptables -A INPUT -p tcp --tcp-flags SYN,FIN SYN,FIN -j DROP

# 7. iptables: rate limiting SYN (max 10/sec)
iptables -A INPUT -p tcp --syn -m limit --limit 10/s --limit-burst 20 -j ACCEPT
iptables -A INPUT -p tcp --syn -j LOG --log-prefix "SYN LIMIT: "
iptables -A INPUT -p tcp --syn -j DROP

# 8. Applica sysctl
sysctl -p

echo "[✓] Hardening applicato."
echo "[!] Ricordare di testare la connettività SSH prima di chiudere la sessione."
```

Fonte: nmap.org/book/man-bypass-firewalls-ids.html; suricata.io/docs/; snort.org/documents; netfilter.org (iptables/nftables); fail2ban.org; [knockd \(zeroflux.org/projects/knock\)](http://knockd.zeroflux.org/projects/knock)

12. CASI D'USO PRATICI E CHEATSHEET

12.1 Workflow di audit di rete completo

■ BASH | full_audit.sh — audit di rete completo in 5 fasi progressive

```
#!/bin/bash
# full_audit.sh – Audit di rete completo in 5 fasi
# Uso: sudo ./full_audit.sh 192.168.1.0/24
# Requisiti: nmap, xsltproc (per HTML), sudo

TARGET="${1:-192.168.1.0/24}"
DATE=$(date +%Y%m%d_%H%M)
DIR="audit_${DATE}"
mkdir -p "$DIR"

echo "=====
echo "  AUDIT DI RETE COMPLETO"
echo "  Target: $TARGET"
echo "  Output: $DIR/"
echo "=====

# FASE 1: Host discovery
echo -e "
[1/5] Host discovery..."
sudo nmap -sn -T4 -PE -PS22,80,443 -PA80 --min-hostgroup 64 -oA "$DIR/fase1_discovery" "$TARGET"
grep "Up" "$DIR/fase1_discovery.gnmap" | awk '{print $2}' > "$DIR/hosts_up.txt"
echo "    Host attivi: $(wc -l < "$DIR/hosts_up.txt")"

# FASE 2: Port scan veloce (top 1000)
echo "[2/5] Port scan veloce top-1000..."
sudo nmap -sS -T4 --min-rate 300 --max-retries 2 -iL "$DIR/hosts_up.txt" -oA "$DIR/fase2_portscan"

# FASE 3: Full port scan su host con porte aperte interessanti
echo "[3/5] Full scan sulle porte critiche..."
sudo nmap -sS -sU -T3 -p T:1-65535,U:53,67,69,123,161,162,500 --max-retries 1 -iL "$DIR/hosts_up.txt" -oA "$DIR/fase3_fullscan"

# FASE 4: Version + OS detection + script default
echo "[4/5] Version detection + OS fingerprinting..."
sudo nmap -sV -O -sC -T4 --osscan-guess --version-intensity 7 -iL "$DIR/hosts_up.txt" -oA "$DIR/fase4_detailed"

# FASE 5: Vulnerability scan
echo "[5/5] Vulnerability scan..."
sudo nmap -sV --script "vuln and not intrusive" -T3 -iL "$DIR/hosts_up.txt" -oA "$DIR/fase5_vuln"

# Riepilogo finale
echo -e "
=====
echo "  RIEPILOGO AUDIT COMPLETATO"
echo "=====
echo "  Host attivi:    $(wc -l < "$DIR/hosts_up.txt")"
echo "  Porte aperte:   $(grep -c "open" "$DIR/fase2_portscan.gnmap" 2>/dev/null || echo 0)"
echo "  Output XML:     $DIR/fase*.xml"
echo "  Output normale: $DIR/fase*.nmap"
echo ""
echo "  Visualizza risultati:"
echo "  cat $DIR/fase4_detailed.nmap"
echo "  cat $DIR/fase5_vuln.nmap | grep -A5 VULNERABLE"
```

12.2 Cheatsheet: comandi per scenario

Scenario	Comando Nmap
Scopri host attivi in una rete	<code>sudo nmap -sn -T4 192.168.1.0/24</code>
Scan veloce top-100 porte	<code>nmap -F 192.168.1.1</code>
Scan completo top-1000 porte con versioni	<code>sudo nmap -sS -sV -T4 192.168.1.1</code>
Tutte le 65535 porte TCP	<code>sudo nmap -p- -T4 --min-rate 1000 192.168.1.1</code>
UDP: servizi DNS, SNMP, DHCP	<code>sudo nmap -sU --top-ports 100 192.168.1.1</code>
OS detection completo	<code>sudo nmap -O --osscan-guess 192.168.1.1</code>
Audit completo (-A = OS+version+script+tracciamento)	<code>sudo nmap -A -T4 192.168.1.1</code>
Solo porte aperte + versioni + script default	<code>sudo nmap -sV -sC --open 192.168.1.1</code>
Scan lento/stealth (elude IDS)	<code>sudo nmap -sS -T1 --scan-delay 2s 192.168.1.1</code>
Trova servizi SMB vulnerabili (EternalBlue)	<code>sudo nmap -p 445 --script smb-vuln-ms17-010 192.168.1.0/24</code>
Verifica Heartbleed su tutti i server HTTPS	<code>sudo nmap -p 443 --script ssl-heartbleed 192.168.1.0/24</code>
Trova FTP anonimi	<code>nmap -p 21 --script ftp-anon 192.168.1.0/24</code>
Enumera condivisioni SMB	<code>nmap -p 445 --script smb-enum-shares,smb-os-discovery 192.168.1.1</code>
Informazioni SSL/TLS e cipher suite	<code>nmap -p 443 --script ssl-cert,ssl-enum-ciphers 192.168.1.1</code>
SNMP: leggi info sistema	<code>sudo nmap -sU -p 161 --script snmp-info 192.168.1.1</code>
Scan con decoy (elude tracciamento)	<code>sudo nmap -D RND:10 -T3 192.168.1.1</code>
Frammenta pacchetti (elude firewall)	<code>sudo nmap -f -T3 192.168.1.1</code>
Usa porta sorgente 53 (DNS)	<code>sudo nmap -g 53 -sS 192.168.1.1</code>
Salva tutto in tutti i formati	<code>sudo nmap -oA output_scan 192.168.1.1</code>
Confronta due scan (trovare differenze)	<code>ndiff scan_lun.xml scan_ven.xml</code>

12.3 Cheatsheet NSE: script per categoria

Categoria	Script chiave	Comando esempio
Web/HTTP	http-title, http-headers, http-methods, http-robots	nmap -p80 --script http-title,http-headers target
SSL/TLS	ssl-cert, ssl-enum-ciphers, ssl-heartbleed	nmap -p443 --script ssl-cert,ssl-enum-ciphers target
SMB/Windows	smb-os-discovery, smb-enum-shares, smb-vuln-*	nmap -p445 --script smb-vuln-* target
SSH	ssh-hostkey, ssh2-enum-algos, ssh-auth-methods	nmap -p22 --script ssh2-enum-algos target
Database	mysql-info, mysql-empty-password, postgresql-brute	nmap -p3306 --script mysql-info,mysql-empty-password target
DNS	dns-brute, dns-zone-transfer, dns-recursion	nmap -p53 --script dns-zone-transfer target
SNMP	snmp-info, snmp-sysdescr, snmp-brute	sudo nmap -sU -p161 --script snmp-info target
FTP	ftp-anon, ftp-bounce, ftp-syst	nmap -p21 --script ftp-anon target
Vulnerabilità,vuln (categoria completa)		nmap --script vuln target
OT/ICS	s7-info, modbus-discover, bacnet-info	nmap -p102 --script s7-info target

13. GLOSSARIO E RIFERIMENTO RAPIDO

13.1 Glossario dei termini di rete e sicurezza

ACK scan — Tecnica Nmap (-sA) che invia pacchetti TCP con solo flag ACK. Non determina se le porte sono aperte ma mappatura regole firewall: RST = non filtrata; nessuna risposta = filtrata.

Banner grabbing — Tecnica che cattura il messaggio di benvenuto di un servizio di rete per identificarne il software e la versione.

CIDR — Classless Inter-Domain Routing. Notazione per specificare range IP: 192.168.1.0/24 = 256 indirizzi dalla .0 alla .255.

CVE — Common Vulnerabilities and Exposures. Identificatore univoco standardizzato per vulnerabilità software. Formato: CVE-ANNO-NUMERO.

Decoy scan — Tecnica Nmap (-D) che mescola pacchetti con IP sorgente falsificati per rendere difficile identificare il reale scanner.

Fingerprinting — Processo di identificazione di caratteristiche uniche di un sistema (OS, applicazione, versione) analizzando le sue risposte di rete.

Half-open scan — Sinonimo di SYN scan: la connessione TCP non viene completata (nessun terzo passo ACK). Meno tracce nei log applicativi.

IDS/IPS — Intrusion Detection/Prevention System. Sistema che monitora e/o blocca traffico sospetto come le scansioni di rete.

NSE — Nmap Scripting Engine. Framework integrato in Nmap che permette di eseguire script Lua per automazioni avanzate.

OS fingerprinting — Identificazione del sistema operativo analizzando le caratteristiche dello stack TCP/IP (TTL, finestra TCP, opzioni TCP, IP ID).

Port knocking — Tecnica di sicurezza che richiede di 'bussare' a una sequenza specifica di porte chiuse per sbloccare una porta (es. SSH).

RAW socket — Tipo di socket che permette di costruire pacchetti di rete arbitrari, bypassando lo stack TCP/IP del sistema operativo. Richiede privilegi root/admin.

RTT — Round-Trip Time. Tempo che impiega un pacchetto per andare all'host target e tornare. Usato da Nmap per calcolare i timeout.

SYN scan — Tecnica di scansione TCP che invia solo il primo pacchetto SYN del three-way handshake. Porta aperta: risponde SYN+ACK; chiusa: RST; filtrata: nessuna risposta.

TCP flags — Bit di controllo nell'header TCP: SYN (connessione), ACK (conferma), FIN (chiusura), RST (reset), PSH (push dati), URG (urgente).

Three-way handshake — Procedura di apertura connessione TCP in tre passi: SYN → SYN+ACK → ACK. Nmap sfrutta le risposte di questi pacchetti.

TTL — Time To Live. Valore nell'header IP decrementato a ogni hop. Valore iniziale tipico: Linux=64, Windows=128, Cisco=255.

UDP scan — Scansione su protocollo UDP (-sU). Più lenta del TCP perché UDP è connectionless e molte porte non rispondono.

Zombie host — Host con IP prevedibile usato nell'idle scan (-sI) come intermediario per una scansione completamente anonima.

Zenmap — GUI grafica ufficiale per Nmap. Permette di costruire comandi visualmente, salvare profili e visualizzare risultati in modo tabellare.

13.2 Opzioni Nmap: riferimento completo

Opzione	Descrizione
-sS	TCP SYN scan (default con root) — stealth, veloce
-sT	TCP Connect scan (no root) — completa l'handshake
-sU	UDP scan — lento ma trova DNS, SNMP, DHCP
-sN/-sF/-sX	NULL/FIN/XMAS scan — elude firewall stateless
-sA	ACK scan — mappa regole firewall
-sI zombie	Idle scan — scansione anonima tramite zombie
-sV	Version detection — identifica software e versione
-O	OS detection — identifica sistema operativo
-A	Aggressive: -O -sV -sC --traceroute
-sC	Script default (categoria 'default')
--script nome	Esegui script NSE specifico
-p porte	Specifica porte: -p 22,80 / -p 1-1024 / -p- (tutte)
-F	Fast scan: top 100 porte
--top-ports N	Scansiona le N porte più comuni
-T0/-T5	Timing: paranoid (lentissimo) a insane (velocissimo)
--min-rate N	Minimo N pacchetti/secondo
--scan-delay T	Pausa T tra probe (es: 1s, 500ms)
-Pn	Salta host discovery — tratta target come up
-n	Nessuna risoluzione DNS
-v/-vv	Verbose / Very verbose
-d/-d5	Debug / Massimo debug
-oN file	Output normale su file
-oX file	Output XML su file
-oG file	Output Grepable su file
-oA base	Tutti i formati contemporaneamente
-D decoy	Usa IP decoy per mascherare scanner
-f	Frammenta pacchetti (8 byte)
--mtu N	MTU personalizzato (multiplo di 8)
-g porta	Porta sorgente fissa (es: -g 53)
--data-length N	Aggiunge N byte casuali ai pacchetti
--spoof-mac	Spoofing MAC address (solo LAN)

Opzione	Descrizione
--ttl N	Imposta TTL personalizzato
-iL file	Leggi target da file
--exclude IP	Escludi IP dalla scansione
--open	Mostra solo porte aperte
--reason	Mostra perché una porta è in quello stato
--packet-trace	Mostra tutti i pacchetti inviati/ricevuti
--resume file	Riprendi scan interrotta da file .gnmap

13.3 Risorse ufficiali e formazione

Risorsa	URL	Contenuto
Nmap Official Site	nmap.org	Download, documentazione, changelog
Nmap Book (online gratuito)	nmap.org/book	Libro completo su Nmap (Fyodor)
NSE Documentation	nmap.org/nsedoc	Reference di tutti i 604+ script NSE
Nmap Man Page	nmap.org/book/man.html	Manuale completo di tutte le opzioni
Nmap GitHub	github.com/nmap/nmap	Codice sorgente e issue tracker
SecTools.org	sectools.org	Top 125 strumenti di sicurezza (include Nmap)
Hack The Box	hackthebox.com	Lab pratici per usare Nmap in CTF
TryHackMe Nmap	tryhackme.com/room/furthernmap	Corso Nmap interattivo gratuito
VulnHub	vulnhub.com	VM vulnerabili per praticare Nmap
OWASP	owasp.org	Guide sicurezza web — complementa Nmap
Exploit-DB	exploit-db.com	Database exploit per CVE trovati con Nmap
NVD NIST	nvd.nist.gov	Database CVE ufficiale per correlazione scan
Shodan	shodan.io	OSINT complementare a Nmap per ricerca pubblica

"Nmap è come un coltellino svizzero per la sicurezza di rete: non fa una sola cosa, ma fa molte cose e le fa bene. Impararlo bene significa imparare come funziona davvero TCP/IP."

Fonte: nmap.org/book (Fyodor Vaskovich 'Nmap Network Scanning'); nmap.org/nsedoc; nmap.org/book/man.html; hackthebox.com; tryhackme.com; vulnhub.com

14. LABORATORI PRATICI

I laboratori pratici sono il modo migliore per imparare Nmap senza rischi. Questa sezione descrive come configurare ambienti di test sicuri e propone esercizi progressivi su target autorizzati e macchine virtuali.

14.1 Target di test ufficiali

Target	URL / IP	Descrizione	Autorizzazione
scanme.nmap.org	45.33.32.156	Server ufficiale Nmap per test pubblici	Sì; — uso moderato
Hack The Box	hackthebox.com	VM vulnerabili con VPN — CTF	Sì; — registrazione
TryHackMe	tryhackme.com	Lab guidati con VM vulnerabili	Sì; — registrazione gratuita
VulnHub	vulnhub.com	VM da scaricare e usare in locale	Sì; — uso locale
DVWA	github.com/digininja/DVWA	Damn Vulnerable Web App — Docker	Sì; — uso locale
Metasploitable 2/3	rapid7.com	VM vulnerabile ufficiale Rapid7	Sì; — uso locale/lab
Kali Purple Targets	kali.org/docs	VM target per Blue Team training	Sì; — uso locale

14.2 Setup laboratorio con Docker

[illegible]

14.3 Esercizi guidati su scanme.nmap.org

■ **INFO** scanme.nmap.org (IP: 45.33.32.156) è un server ufficiale Nmap disponibile per test pubblici con uso moderato. Non usarlo per scan aggressivi o ripetuti automaticamente.

■ BASH | Esercizi progressivi su scanme.nmap.org — target di test ufficiale

```
# ■■■ ESERCIZI SU scanme.nmap.org ■■■■■■  
TARGET="scanme.nmap.org"    # = 45.33.32.156  
  
# Esercizio 1: Scan base  
nmap $TARGET  
# Domanda: quante porte aperte? Quali servizi?  
  
# Esercizio 2: Versioni servizi  
nmap -sV $TARGET  
# Domanda: quale versione di SSH? Quale server web?  
  
# Esercizio 3: OS detection  
sudo nmap -O $TARGET  
# Domanda: quale sistema operativo?  
  
# Esercizio 4: Script NSE default  
nmap -sC $TARGET  
# Domanda: cosa riporta lo script ssh-hostkey?  
  
# Esercizio 5: Tutte le porte  
sudo nmap -p- -T4 --min-rate 1000 $TARGET  
# Domanda: ci sono servizi su porte non standard?  
  
# Esercizio 6: UDP scan (lento)  
sudo nmap -sU --top-ports 20 $TARGET  
# Domanda: ci sono servizi UDP attivi?  
  
# Esercizio 7: Traceroute  
nmap --traceroute $TARGET  
# Domanda: quanti hop per raggiungere scanme?  
  
# Esercizio 8: Output completo in tutti i formati  
sudo nmap -A -oA esercizio_scanme $TARGET  
wc -l esercizio_scanme.nmap    # quante righe?  
xmllint --format esercizio_scanme.xml | head -30 # struttura XML
```

14.4 CTF con Nmap: approccio metodico

Le CTF (Capture The Flag) sono competizioni di sicurezza informatica in cui si attaccano macchine vulnerabili per trovare 'flag' nascoste. Nmap è quasi sempre il primo strumento usato per la fase di ricognizione:

■ BASH | ctf_recon.sh — ricognizione automatizzata per CTF con Nmap

```
#!/bin/bash
# ctf_recon.sh – Ricognizione iniziale per CTF
# Uso: ./ctf_recon.sh 10.10.10.1

TARGET="${1:-10.10.10.1}"
echo "=== CTF RECON: $TARGET ==="

# Step 1: Ping (il target e' up?)
echo -n "[1] Ping... "
if ping -c1 -W1 "$TARGET" &>/dev/null; then
    echo "UP"
else
    echo "DOWN (prova con -Pn)"
fi

# Step 2: Top 1000 porte rapido (primo sguardo)
echo "[2] Top-1000 porte (veloce)..."
sudo nmap -sS -T4 --open "$TARGET" 2>/dev/null | grep -E "[0-9]+/tcp"

# Step 3: Tutte le porte (piu lento ma completo)
echo "[3] Full port scan (tutte 65535)..."
OPEN_PORTS=$(sudo nmap -p- -T4 --min-rate 1000 --open "$TARGET" 2>/dev/null
' ', ' | sed 's/,,$//')
echo "    Porte aperte: $OPEN_PORTS"

# Step 4: Version + script sulle porte trovate
if [ -n "$OPEN_PORTS" ]; then
    echo "[4] Version + script detection su porte: $OPEN_PORTS"
    sudo nmap -sV -sC -O -p "$OPEN_PORTS" --script "default,vuln" -oN "ctf_${TARGET}"
    echo "[✓] Salvato: ctf_${TARGET}.nmap"
fi

# Step 5: Riepilogo rapido
echo ""
echo "=== RIEPILOGO ==="
echo "Porte: $OPEN_PORTS"
grep "Service Info\|OS:\|Running:" "ctf_${TARGET}.nmap" 2>/dev/null | head -5
```

Fonte: scanme.nmap.org; hackthebox.com; tryhackme.com; vulnhub.com; github.com/digininja/DVWA; hub.docker.com/r/tleemcjr/metasploitable2

15. NMAP IN AMBIENTI ENTERPRISE E COMPLIANCE

In ambienti aziendali Nmap viene usato sistematicamente per l'asset discovery, il vulnerability management, la verifica della compliance (PCI-DSS, ISO 27001, NIS2) e il monitoraggio continuo della superficie di attacco.

15.1 Asset inventory automatizzato

■ PYTHON | asset_inventory.py — inventario automatico con SQLite e python-nmap

```
#!/usr/bin/env python3
"""
asset_inventory.py
Genera inventario automatico di tutti gli asset di rete.
Aggiorna un database SQLite con i risultati di ogni scan.
Dipendenze: pip install python-nmap
"""

import nmap
import sqlite3
import json
from datetime import datetime

DB_FILE = "asset_inventory.db"

def init_db(conn):
    conn.executescript("""
CREATE TABLE IF NOT EXISTS assets (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    ip            TEXT NOT NULL,
    hostname      TEXT,
    mac           TEXT,
    os            TEXT,
    first_seen    TEXT,
    last_seen     TEXT,
    UNIQUE(ip)
);
CREATE TABLE IF NOT EXISTS services (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    asset_id      INTEGER REFERENCES assets(id),
    port          INTEGER,
    protocol      TEXT,
    state         TEXT,
    service       TEXT,
    product       TEXT,
    version       TEXT,
    last_seen     TEXT
);
CREATE TABLE IF NOT EXISTS scan_history (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp     TEXT,
    target        TEXT,
    command       TEXT,
    hosts_up      INTEGER,
    duration_s    REAL
);
""")
    conn.commit()

def update_inventory(target: str, scan_args: str = "-sS -sV -O -T4"):
    nm = nmap.PortScanner()
    start = datetime.now()
    print(f"[*] Scanning {target}...")
    nm.scan(hosts=target, arguments=scan_args, sudo=True)
    elapsed = (datetime.now() - start).total_seconds()
    conn = sqlite3.connect(DB_FILE)
```

```

init_db(conn)
now = datetime.now().isoformat()

hosts_up = 0
for host in nm.all_hosts():
    if nm[host].state() != 'up':
        continue
    hosts_up += 1
    hostname = nm[host].hostname()
    mac = nm[host].get('addresses', {}).get('mac', '')

    # OS detection
    os_name = ''
    if 'osmatch' in nm[host] and nm[host]['osmatch']:
        os_name = nm[host]['osmatch'][0].get('name', '')

    # Upsert asset
    conn.execute("""
        INSERT INTO assets (ip, hostname, mac, os, first_seen, last_seen)
        VALUES (?, ?, ?, ?, ?, ?)
        ON CONFLICT(ip) DO UPDATE SET
            hostname=excluded.hostname,
            mac=excluded.mac,
            os=excluded.os,
            last_seen=excluded.last_seen
    """, (host, hostname, mac, os_name, now, now))

    asset_id = conn.execute(
        "SELECT id FROM assets WHERE ip=?", (host,)
    ).fetchone()[0]

    # Servizi
    for proto in nm[host].all_protocols():
        for port in nm[host][proto]:
            p = nm[host][proto][port]
            if p['state'] != 'open':
                continue
            conn.execute("""
                INSERT INTO services
                (asset_id, port, protocol, state, service, product, version, last_seen)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?)
            """, (asset_id, port, proto, p['state'],
                p.get('name', ''), p.get('product', ''),
                p.get('version', ''), now))

    # Log scan
    conn.execute("""
        INSERT INTO scan_history (timestamp, target, command, hosts_up, duration_s)
        VALUES (?, ?, ?, ?, ?)
    """, (now, target, nm.command_line(), hosts_up, elapsed))

    conn.commit()
    conn.close()
    print(f"[✓] Inventario aggiornato: {hosts_up} host | {elapsed:.1f}s")

def print_inventory():

```



```

conn = sqlite3.connect(DB_FILE)
print("
=== ASSET INVENTORY ===")
assets = conn.execute("""
    SELECT a.ip, a.hostname, a.os, a.last_seen,
           COUNT(s.id) as open_ports
    FROM assets a
    LEFT JOIN services s ON s.asset_id = a.id
    GROUP BY a.id
    ORDER BY a.ip
""").fetchall()

for ip, hostname, os_name, last_seen, ports in assets:
    print(f"
{ip:16s} {hostname or '':20s} porte: {ports}")
    print(f"  OS: {(os_name or 'N/A')[:50]}")
    print(f"  Ultimo scan: {last_seen[:19]}")
conn.close()

if __name__ == "__main__":
    import sys
    target = sys.argv[1] if len(sys.argv) > 1 else "192.168.1.0/24"
    update_inventory(target)
    print_inventory()

```

15.2 Monitoraggio continuo: rilevare nuovi host e servizi

■ PYTHON | network_monitor.py — monitoraggio continuo con alert email

```
#!/usr/bin/env python3
"""
network_monitor.py
Monitora la rete e avvisa quando appaiono nuovi host o servizi.
Invia alert via email quando vengono rilevate anomalie.
"""

import nmap
import json
import smtplib
import time
from email.mime.text import MIMEText
from pathlib import Path
from datetime import datetime

STATE_FILE = "network_state.json"
CHECK_INTERVAL = 3600 # 1 ora

def load_state() -> dict:
    if Path(STATE_FILE).exists():
        with open(STATE_FILE) as f:
            return json.load(f)
    return {}

def save_state(state: dict):
    with open(STATE_FILE, 'w') as f:
        json.dump(state, f, indent=2)

def scan_network(target: str) -> dict:
    nm = nmap.PortScanner()
    nm.scan(hosts=target, arguments="-sS -sV -T4 --top-ports 100", sudo=True)
    current = {}
    for host in nm.all_hosts():
        if nm[host].state() != 'up':
            continue
        ports = {}
        for proto in nm[host].all_protocols():
            for port in nm[host][proto]:
                p = nm[host][proto][port]
                if p['state'] == 'open':
                    key = f"{port}/{proto}"
                    ports[key] = f"{p.get('product', '')} {p.get('version', '')}".strip()
        current[host] = {
            "hostname": nm[host].hostname(),
            "ports": ports,
            "last_seen": datetime.now().isoformat()
        }
    return current

def find_changes(old: dict, new: dict) -> list:
    changes = []
    # Nuovi host
    for host in new:
        if host not in old:
            changes.append(f"NUOVO HOST: {host} ({new[host]['hostname']})")
            for p, svc in new[host]['ports'].items():
```

```

        changes.append(f" -> Porta: {p} {svc}")
# Nuove porte su host esistenti
for host in new:
    if host in old:
        for port in new[host]['ports']:
            if port not in old[host]['ports']:
                svc = new[host]['ports'][port]
                changes.append(f"NUOVA PORTA: {host}:{port} {svc}")
# Host spariti
for host in old:
    if host not in new:
        changes.append(f"HOST SPARITO: {host} ({old[host].get('hostname', '')})")
return changes

def send_alert(changes: list, smtp_server: str = "localhost"):
    msg = MIMEText("
".join(changes))
    msg['Subject'] = f"[ALERT] Network Monitor: {len(changes)} cambiamenti rilevati"
    msg['From'] = "nmap-monitor@company.local"
    msg['To'] = "security@company.local"
    try:
        with smtplib.SMTP(smtp_server) as s:
            s.send_message(msg)
            print("[✓] Alert email inviata")
    except Exception as e:
        print(f"[!] Errore email: {e}")

def monitor_loop(target: str):
    print(f"[*] Avvio monitoraggio: {target} (ogni {CHECK_INTERVAL}s)")
    while True:
        print(f"
[{{datetime.now():%H:%M:%S}}] Scan in corso...")
        old_state = load_state()
        new_state = scan_network(target)
        changes = find_changes(old_state, new_state)

        if changes:
            print(f"[!] {len(changes)} cambiamenti rilevati!")
            for c in changes:
                print(f" {c}")
            send_alert(changes)
        else:
            print("[OK] Nessun cambiamento")

        save_state(new_state)
        time.sleep(CHECK_INTERVAL)

if __name__ == "__main__":
    import sys
    target = sys.argv[1] if len(sys.argv) > 1 else "192.168.1.0/24"
    monitor_loop(target)

```

15.3 Nmap e framework di compliance

Standard	Requisito rilevante	Uso di Nmap
PCI-DSS v4	Req. 11.2: scansione vulnerabilità Perimetrale NSE e CDE	Perimetrale NSE e CDE su tutti i sistemi in scope
PCI-DSS v4	Req. 11.5: deployment IDS/IPS	Nmap usato per verificare efficacia IDS con scan controllati
ISO 27001	A.12.6: gestione vulnerabilità tecniche	Scans periodici per asset inventory e rilevamento servizi non autorizzati
NIS2 (EU)	Art. 21: misure di sicurezza per operatori	Sessioni continue per identificare esposizioni su infrastrutture critiche
GDPR	Art. 32: misure tecniche adeguate	Asset discovery per verificare che solo servizi autorizzati siano esposti
CIS Controls v8	Control 7: gestione vulnerabilità Scans	Scans settimanali con Nmap integrato in workflow di remediation
SOC 2 Type II	Availability + Security: monitoring continuo	Nmap integrato in pipeline CI/CD per verifica post-deploy
NIST CSF	ID.AM: asset management; DE.CM: monitoraggio continuo	Baseline inventory con Nmap + alerting su cambiamenti

✓ **BEST PRACTICE** Per ambienti compliance: documentare ogni scansione con timestamp, scope, responsabile e risultati. Conservare i file XML per 12-24 mesi come evidenza per audit. Usare Ndiff per produrre report di variazione tra scan consecutivi.

15.4 Integrazione in pipeline CI/CD

■ YAML | GitHub Actions: integrazione Nmap in pipeline CI/CD post-deploy

[illegible]

```
name: "Security Scan Post-Deploy"
```

on :

```
deployment_status:
  # Esegue solo quando il deployment e' completato con successo
```

jobs:

```
nmap_scan:
  if: github.event.deployment_status.state == 'success'
  runs-on: ubuntu-latest
  permissions:
    issues: write
```

steps:

```
- name: Install Nmap
  run: sudo apt-get install -y nmap
```

- name: Run Nmap security scan

id: scan

```
run: |
    TARGET="${{ secrets.STAGING_HOST }}"
    echo "Scanning: $TARGET"
```

```
# Scan porte aperte
```

```
sudo nmap -sS -sV -T4 --open -oX scan_result.xml
```

-p 1-65535 "\$"

Conta porte aperte

```
OPEN=$(grep -c 'state="open"' scan_result.xml || echo 0)
echo "open_ports=$OPEN" >> $GITHUB_OUTPUT
```

```
# Verifica porte NON autorizzate
```

UNAUTH=""

```
ALLOWED_PORTS="22 80 443 8080"
```

```
while IFS= read -r line; do
```

```
if echo "$line" | grep -q 'state="open"'; then
```

```
PORT=$(echo "$line" | grep -oP 'portid="\K[0-9]+')
```

```
if ! echo "$ALLOWED_PORTS" | grep -qw "$PORT"; then
```

UNAUTH="\$UNAUTH \$PORT"

fi

fi

```
done < scan_result.xml
```

```
if [ -n "$SUNAUTH" ]; then
```

```
echo "unauthorized_ports=$UNAUTH" >> $GITHUB_OUTPUT
```

```
echo "[!] Porte NON autorizzate trovate: $UNAUTH"
```

```
exit 1    # fail il workflow
```

fi

```
- name: Create Issue for unauthorized ports
```

```
if: failure()
```

```
uses: actions/github-script@v7
```

```
with:
  script: |
    github.rest.issues.create({
      owner: context.repo.owner,
      repo: context.repo.repo,
      title: "[SECURITY] Porte non autorizzate rilevate post-deploy",
      body: "Trovate porte non autorizzate: " +
        "${{ steps.scan.outputs.unauthorized_ports }}\n\n" +
        "Host: ${{ secrets.STAGING_HOST }}\n" +
        "Run: ${{ github.server_url }}/${{ github.repository }}/actions/runs/${{ github.run_id }}",
      labels: ["security", "bug"]
    })
```

Fonte: PCI Security Standards Council pci-ssc.com; ISO/IEC 27001:2022; EUR-Lex NIS2 (eur-lex.europa.eu/legal-content/IT/TXT/?uri=CELEX:32022L2555); CIS Controls v8 (cisecurity.org/controls); NIST Cybersecurity Framework (nist.gov/cyberframework)

16. APPENDICE: COMANDI AVANZATI E TABELLE

16.1 Comandi Nmap avanzati meno noti

■ BASH | Comandi Nmap avanzati: IPv6, Ndiff, Nping, Ncat, resume, XSLT

[illegible]

[illegible]

16.2 Tabella porte comuni e servizi

Porta	Proto	Servizio	Informazioni utili per audit
21	TCP	FTP	Verifica anonimo: <code>--script ftp-anon</code> ; cleartext = insicuro
22	TCP	SSH	Verifica algoritmi: <code>--script ssh2-enum-algos</code> ; versione obsoleta?
23	TCP	Telnet	Sempre insicuro — cleartext; dovrebbe essere disabilitato
25	TCP	SMTP	Open relay? <code>--script smtp-open-relay</code> ; autenticazione?
53	TCP/UDP	DNS	Zone transfer: <code>--script dns-zone-transfer</code> ; recursion?
67/68	UDP	DHCP	Server DHCP non autorizzati: rogue DHCP risk
69	UDP	TFTP	Non autenticato — espone file system; raramente necessario
80	TCP	HTTP	Web server: titolo, tecnologie, redirect a HTTPS?
110	TCP	POP3	Email cleartext — usare POP3S (995)
123	UDP	NTP	Amplification DDoS risk; version: <code>--script ntp-info</code>
143	TCP	IMAP	Email cleartext — usare IMAPS (993)
161	UDP	SNMP	Community string 'public': <code>--script snmp-info</code>
389	TCP	LDAP	Directory service; anonymous bind? NULL base DN?
443	TCP	HTTPS	SSL/TLS: <code>ssl-cert</code> , <code>ssl-enum-ciphers</code> , <code>ssl-heartbleed</code>
445	TCP	SMB	Vulnerabilità critiche: <code>smb-vuln-ms17-010</code> (EternalBlue)
502	TCP	Modbus	OT protocol — nessuna autenticazione nativa!
514	UDP	Syslog	Cleartext — sniff dei log possibile
1433	TCP	MSSQL	<code>--script ms-sql-info</code> , <code>ms-sql-empty-password</code>
1521	TCP	Oracle DB	<code>--script oracle-tns-version</code> , <code>oracle-brute</code>
1883	TCP	MQTT	IoT broker — autenticazione? <code>--script mqtt-info</code>
2375	TCP	Docker API	Non autenticato = RCE sul host! Non esporre mai
3306	TCP	MySQL	<code>--script mysql-empty-password</code> , <code>mysql-databases</code>
3389	TCP	RDP	BlueKeep: <code>rdp-vuln-ms12-020</code> ; NLA attiva?
4840	TCP	OPC-UA	OT/SCADA; <code>--script opc-ua-discovery</code>
5432	TCP	PostgreSQL	<code>--script pgsq-brute</code> ; <code>pg_hba.conf</code> configurato?
5900	TCP	VNC	Spesso senza password; <code>--script realvnc-auth-bypass</code>
6379	TCP	Redis	Accesso senza auth: <code>--script redis-info</code>
8080	TCP	HTTP-alt	Web app su porta alternativa; stesso check di 80
8443	TCP	HTTPS-alt	Stesso check di 443
27017	TCP	MongoDB	Senza auth: <code>--script mongodb-info</code>
47808	UDP	BACnet	Building automation — <code>--script bacnet-info</code>

Fonte: IANA Port Numbers Registry (iana.org/assignments/port-numbers); nmap.org/nsedoc; ICS-CERT (cisa.gov/ics); Shodan Trends (shodan.io/trends)

17. RIEPILOGO, CONFRONTI E CONCLUSIONI

17.1 Nmap vs altri scanner: confronto

Tool	Tipo	Punti di forza	Limiti	Integrare con Nmap?
Nmap	Port scanner + NSE	Versatile, scriptabile, free, standard in industria	Behindustrial; GUI limitata	
Masscan	Port scanner massa	10M+ pacchetti/sec — scansione	Solo IPv4, solo un NSE, Scansione	Scansione OS discovery poi Nmap su host trovati
Zmap	Port scanner massa	Simile a Masscan, ricerca accelerata	Meno feature di Nmap	S` stesso workflow di Masscan
Nessus	Vulnerability scanner	Database CVE enorme, report completo	Applicazione (oltre soglia)	S` import XML Nmap
OpenVAS	Vulnerability scanner	Free, open source, aggiornamenti	Requisiti, installazione complessa	S` import XML Nmap
Nikto	Web scanner	Specializzato HTTP, veloce per	Solo HTTP/HTTPS	S` dopo Nmap per porte web
Metasploit	Exploit framework	1000+ exploit, post-exploitation	Non ` uno scanner	S` import XML Nmap nativamente
Shodan	OSINT internet scanner	Trova device esposti senza scansione	Solo dati pubblici, non real-time	S` integra complementa per OSINT
Rustscan	Port scanner	Ultra-veloce (tutto TCP in <1s)	Meno feature; wrapper	S` discovery poi passa a Nmap

17.2 Script NSE per tipo di servizio: mappa completa

Servizio / Protocollo	Script NSE consigliati	Nota
HTTP/HTTPS	http-title, http-headers, http-methods, http-auth-finder, http-robots.txt, http-git, http-shellshock, http-vuln-*	Usare <code>http-robots.txt</code> per trovare informazioni sui siti web.
SSL/TLS	ssl-cert, ssl-enum-ciphers, ssl-heartbleed, ssl-dh-param, ssl-enum-ciphers, ssl-enum-ciphers, ssl-enum-ciphers	Essere attenti a vulnerabilità, HTTP
SSH	ssh-hostkey, ssh2-enum-algos, ssh-auth-methods, ssh-brute, ssh-hostkey, ssh-brute, ssh-hostkey	SSH è un protocollo per sistemi autorizzati
FTP	ftp-anon, ftp-bounce, ftp-syst, ftp-brute, ftp-libopie	ftp-anon: primo controllo sempre
SMTP/Email	smtp-command, smtp-enum-users, smtp-open-relay, smtp-enum-users, smtp-enum-users	SMTP è un protocollo per email, problema critico
DNS	dns-zone-transfer, dns-brute, dns-recursion, dns-cache-spoof, dns-zone-transfer, dns-zone-transfer	Zone transfer, dns-zone-transfer, dns-zone-transfer
SMB/Windows	smb-os-discovery, smb-enum-shares, smb-enum-users, smb-enum-users, smb-enum-users	Porte 139 e 445, smb-vuln-ms08-067, smb-security-model
RDP	rdp-enum-encryption, rdp-vuln-ms12-020, rdp-enum-encryption, rdp-enum-encryption	BlueKeep separato (CVE-2019-0708)
SNMP	snmp-info, snmp-sysdescr, snmp-interfaces, snmp-pdu, snmp-brute, snmp-brute, snmp-brute	Porte 161 e 162, snmp-brute, snmp-brute
LDAP	ldap-rootdse, ldap-search, ldap-novell-getpass, ldap-anon, ldap-anon, ldap-anon	Anonimous bind test
Database MySQL	mysql-info, mysql-databases, mysql-empty-password, mysql-empty-password, mysql-empty-password	Porte 3306, mysql-empty-password, mysql-brute
Database MSSQL	ms-sql-info, ms-sql-config, ms-sql-empty-password, ms-sql-empty-password, ms-sql-empty-password	Porte 1433, ms-sql-empty-password, ms-sql-xp-cmdshell
MongoDB	mongodb-info, mongodb-databases	Porta 27017 — spesso senza auth
Redis	redis-info, redis-brute	Porta 6379 — accesso spesso aperto
OT/ICS Modbus	modbus-discover	UDP/TCP 502 — nessuna auth nativa
OT/ICS Siemens S7	s7-info	TCP 102 — identifica CPU, versione
OT/ICS BACnet	bacnet-info	UDP 47808 — building automation
OT/ICS OPC-UA	opc-ua-discovery	TCP 4840 — standard IEC
VNC	realvnc-auth-bypass, vnc-info, vnc-brute	TCP 5900 — spesso senza password
Telnet	telnet-encryption, telnet-ntlm-info	TCP 23 — sempre disabilitare

17.3 Workflow decisionale: quale scan usare?

■ BASH | Guida decisionale: come scegliere il comando Nmap giusto per ogni scenario

```

# DOMANDA 1: Hai i privilegi root/admin?
# SI -> Usa -sS (SYN scan) – veloce e relativamente discreto
# NO -> Usa -sT (TCP connect) – piu lento, lascia log

# DOMANDA 2: Quante porte vuoi scansionare?
# Solo le piu comuni -> nmap -F (top 100)
# Standard -> nmap (top 1000, default)
# Tutte -> nmap -p-

# DOMANDA 3: Hai bisogno di versioni dei servizi?
# SI -> aggiungi -sV
# NO -> risparmia tempo

# DOMANDA 4: Hai bisogno di rilevare l OS?
# SI -> aggiungi -O (richiede root)
# NO -> non aggiungerlo

# DOMANDA 5: Reti con IDS/firewall da bypassare?
# Lentezza -T1 -> --scan-delay 2s
# Decoy -> -D RND:10
# Frammentazione -> -f
# Porta sorgente -> -g 53 o -g 80

# DOMANDA 6: Vuoi cercare vulnerabilita specifiche?
# Web -> --script "http-vuln-*"
# SMB -> --script "smb-vuln-*"
# SSL -> --script ssl-heartbleed,ssl-enum-ciphers
# Tutte -> --script vuln (lento!)
# Default -> -sC (script sicuri)

# DOMANDA 7: Stai lavorando su rete OT/ICS?
# SI -> usa -T2 (non sovraccaricare PLC!)
# -> porte specifiche: -p 102,502,4840,20000,44818,47808
# -> script OT: s7-info, modbus-discover, bacnet-info

# ESEMPI RACCOMANDATI PER SCENARIO:

# Audit rapido LAN (10 min)
sudo nmap -sS -sV -T4 --open 192.168.1.0/24

# Audit approfondito singolo host (30 min)
sudo nmap -sS -sU -sV -O -sC -p- -T3 192.168.1.1

# Ricognizione silenziosa (ore)
sudo nmap -sS -T1 --scan-delay 3s -D RND:5 --top-ports 100 192.168.1.1

# Audit OT/ICS (sicuro su rete industriale)
sudo nmap -sS -T2 --max-rate 50 -p 102,502,4840,20000,44818,47808 --script s7-info,modbus-di

# CTF / Hack The Box
sudo nmap -sS -sV -sC -O -p- -T4 --min-rate 1000 10.10.10.X

# Compliance scan per PCI-DSS
sudo nmap -sV --script "vuln" -T3 -oA pci_scan_$(date +%Y%m%d) 192.168.1.0/24

```

17.4 Conclusioni

Nmap è molto più di un semplice port scanner: è una piattaforma di intelligence di rete che, dopo quasi trent'anni, rimane lo strumento di riferimento per chiunque lavori nella sicurezza informatica. La sua forza risiede nella combinazione di velocità, accuratezza, estensibilità tramite NSE e integrazione con tutto l'ecosistema della sicurezza offensiva e difensiva.

Per i professionisti della sicurezza OT e dell'automazione industriale, Nmap rappresenta uno strumento indispensabile per l'inventario degli asset, la verifica dell'esposizione di protocolli industriali (Modbus, S7, OPC-UA, BACnet) e l'audit periodico delle reti di controllo. L'integrazione con Python permette di costruire pipeline automatizzate di monitoraggio continuo conformi alle normative IEC 62443 e NIS2.

La regola d'oro rimane invariata: **usare Nmap solo su sistemi propri o con autorizzazione esplicita**. La conoscenza di questo strumento è un potere — usarlo con responsabilità e nel rispetto delle leggi vigenti è l'unico approccio professionale accettabile.

"Con grande potere deriva grande responsabilità. Nmap è potente: usalo per proteggere, non per attaccare."

Argomento	Capitoli	Risorse
Storia e fondamentali	1, 2, 3	nmap.org/book
Tecniche di scansione	4, 5	nmap.org/book/man-port-scanning-techniques.html
Version detection e OS	6	nmap.org/book/vscan.html
NSE scripting	7	nmap.org/nsedoc
Evasione firewall/IDS	8	nmap.org/book/man-bypass-firewalls-ids.html
Automazione Python	9	xael.org/pages/python-nmap.html
Integrazione Metasploit	10	docs.metasploit.com
Difesa e hardening	11	owasp.org
Lab pratici	14	tryhackme.com/room/furthernmap
Enterprise/Compliance	15	cisecurity.org/controls

Fonte: Fyodor 'Nmap Network Scanning' (nmap.org/book); nmap.org/nsedoc; nmap.org/book/man.html; tryhackme.com; hackthebox.com; vulnhub.com; cisecurity.org; nist.gov/cyberframework

18. TECNICHE NSE AVANZATE E CASI REALI

Questa sezione approfondisce l'uso di NSE in scenari reali di penetration test e security audit, mostrando combinazioni di script e workflow completi per le fasi di reconnaissance e vulnerability assessment.

18.1 Ricognizione web completa con NSE

■ BASH | Web recon completo: tecnologie, directory, WAF, vulnerabilità, SSL

[illegible]

18.2 Enumerazione Active Directory / SMB

■ BASH | Enumerazione Active Directory: SMB, LDAP, Kerberos, NetBIOS

[illegible]

18.3 Audit infrastruttura cloud e container

■ BASH | Audit cloud e container: Docker, Kubernetes, Elasticsearch, Redis

[illegible]

18.4 Brute force controllato con NSE

■ **ATTENZIONE** Gli script di brute force devono essere usati SOLO su sistemi propri o con autorizzazione scritta esplicita. Il brute force non autorizzato costituisce reato penale in tutte le giurisdizioni.

■ BASH | Brute force controllato con NSE: SSH, FTP, HTTP, MySQL, SNMP

■■ BRUTE FORCE CON NMAP NSE (solo sistemi autorizzati) ■■■■■■■■■■

Preparazione: crea file wordlist

cat > users.txt << 'EOF'

admin

administrator

root

user

test

guest

EOF

cat > pass.txt << 'EOF'

admin

password

123456

password123

admin123

test

EOF

SSH brute force

nmap -p 22 --script ssh-brute --script-args userdb=users.txt,passdb=pass.txt

--script-args s

FTP brute force

nmap -p 21 --script ftp-brute --script-args userdb=users.txt,passdb=pass.txt

192.168.1.1

HTTP Basic Auth brute force

nmap -p 80 --script http-brute --script-args userdb=users.txt,passdb=pass.txt

--script-args l

MySQL brute force

nmap -p 3306 --script mysql-brute --script-args userdb=users.txt,passdb=pass.txt

192.168.1.1

SNMP community string brute force

nmap -sU -p 161 --script snmp-brute 192.168.1.1

Prova automaticamente le community string piu comuni

Parametri comuni per tutti gli script brute:

brute.mode = user (per ogni user prova tutte le pass)

= pass (per ogni pass prova tutti gli user)

= user+pass (combinazioni)

brute.firstonly = true (ferma al primo match)

brute.delay = 1 (delay in secondi tra tentativi)

Esempio con parametri completi

nmap -p 22 --script ssh-brute --script-args "userdb=users.txt,passdb=pass.txt,brute.mode=use

Fonte: nmap.org/nsedoc/categories/brute.html; nmap.org/nsedoc/scripts/http-enum.html; nmap.org/nsedoc/scripts/smb-enum-shares.html; owasp.org/www-project-web-security-testing-guide/

19. RICETTE RAPIDE E AUTOMAZIONI BASH

Questa sezione raccoglie ricette pratiche, snippet bash pronti all'uso e automazioni per i compiti più comuni dell'amministratore di rete e del security analyst.

19.1 Snippet bash pronti all'uso

■ BASH | Snippet bash pronti all'uso: find, diff, count, parallel, alert

```
#!/bin/bash
# ■■■ SNIPPET 1: Trova tutti i server SSH nella rete ■■■■
echo "=== Server SSH nella rete ==="
sudo nmap -ss -p 22 --open 192.168.1.0/24 | grep "Nmap scan report" | awk '{print $NF}' | tr -d '\n'

# ■■■ SNIPPET 2: Trova tutti i server web (HTTP e HTTPS) ■■■■
echo "=== Web server nella rete ==="
sudo nmap -SS -p 80,443,8080,8443 --open -oG - 192.168.1.0/24 | grep "open" | awk '{print $2, $5}'

# ■■■ SNIPPET 3: Trova servizi con versioni obsolete / EOL ■■■■
echo "=== Versioni software (per verifica EOL) ==="
sudo nmap -sV --version-intensity 5 -oX - 192.168.1.0/24 | grep -E "product|version" | sort -u

# ■■■ SNIPPET 4: Conta porte aperte per host ■■■■
sudo nmap -ss --open -oG - 192.168.1.0/24 | grep "^Host:" | awk '{count = split($0, a, "open/") print $2, (count-1), "porte aperte"}' | sort -k2 -rn

# ■■■ SNIPPET 5: Trova host con piu di 10 porte aperte ■■■■
echo "=== Host con molte porte aperte (possibile honeypot o errore) ==="
sudo nmap -ss --open -oG - 192.168.1.0/24 | awk '/^Host:/ {n = split($0, a, "open")} if (n-1 > 10) print $2, n-1, "porte"}'

# ■■■ SNIPPET 6: Diff rapido tra due scan ■■■■
# Prima scan
sudo nmap -ss -oX scan_prima.xml 192.168.1.0/24
echo "Aspetta 1 settimana, poi:"
# Seconda scan
sudo nmap -ss -oX scan_dopo.xml 192.168.1.0/24
# Confronto
ndiff scan_prima.xml scan_dopo.xml | grep -E "\+|-\" | head -30

# ■■■ SNIPPET 7: Esporta lista IP:PORTA per ogni servizio ■■■■
# Tutti gli host con porta 443 aperta
grep "443/open" scan.gnmap | awk '{print $2":443"}'

# ■■■ SNIPPET 8: Scan parallela su piu reti con GNU parallel ■■■■
# sudo apt install parallel
echo -e "192.168.1.0/24\n10.0.0.0/24\n172.16.0.0/24" | parallel -j3 "sudo nmap -ss -T4 --open -oA scan_{#} {}]"

# ■■■ SNIPPET 9: Alert se nuova porta aperta su host critico ■■■■
PREVIOUS=$(nmap -ss --open 192.168.1.1 | grep "open" | md5sum)
sleep 3600
CURRENT=$(nmap -ss --open 192.168.1.1 | grep "open" | md5sum)
if [ "$PREVIOUS" != "$CURRENT" ]; then
    echo "ALERT: Cambiamento porte su 192.168.1.1!" | mail -s "NMAP ALERT" security@company.local
fi

# ■■■ SNIPPET 10: Generare report HTML da XML con xsltproc ■■■■
sudo nmap -sV -sC -oX report.xml 192.168.1.0/24
```

```
xsltproc /usr/share/nmap/nmap.xsl report.xml -o report.html  
echo "Report: $(pwd)/report.html"
```

19.2 Integrazione con altre utility Unix

■ BASH | Nmap integrato con grep, awk, jq, netcat e loop automatici

[illegible]

```
BEGIN {print "IP,Hostname,Porte_aperte"}
{
    ip = $2
    hostname = substr($3, 2, length($3)-2)
    ports = ""
    for(i=5; i<=NF; i++) {
        if ($i ~ /open/) {
            split($i, a, "/")
            ports = ports a[1] " "
        }
    }
    print ip "," hostname "," ports
}' > report.csv
echo "CSV salvato: report.csv"
```

19.3 Nmap per amministratori di rete OT

Per chi lavora su reti industriali (OT/ICS), Nmap richiede particolare cautela ma rimane uno strumento prezioso per la visibilità degli asset. Ecco le best practice e i comandi specifici:

■ BASH | ot_discovery.sh — inventario OT/ICS sicuro con porte industriali e NSE

```
#!/bin/bash
# ot_discovery.sh – Inventario asset OT/ICS sicuro
# REGOLA FONDAMENTALE: usare -T2 o -T1 su reti OT
# I PLC e RTU hanno stack TCP/IP limitati che possono crashare
# con scan aggressive (-T4/-T5 sono VIETATI su reti produzione OT)

NETWORK="${1:-192.168.100.0/24}"
OUT="ot_scan_$(date +%Y%m%d)"

echo "[!] Avvio scan OT su $NETWORK"
echo "[!] Usando -T2 (polite) per non sovraccaricare dispositivi"

# Fase 1: Host discovery leggero (ARP, no ICMP aggressivo)
echo "[1/4] Host discovery leggero..."
sudo nmap -sn -T2 --send-ip -PE -PS80 --max-retries 1 -oA "${OUT}_discovery" "$NETWORK"

grep "Up" "${OUT}_discovery.gnmap" | awk '{print $2}' > "${OUT}_hosts.txt"
echo "Host trovati: $(wc -l < "${OUT}_hosts.txt")"

# Fase 2: Scan porte OT specifiche (non tutte le porte!)
echo "[2/4] Scan porte OT comuni..."
sudo nmap -sS -T2 --max-rate 30 --max-retries 1 -p 20,21,22,23,80,102,443,502,1089,1090 -oA "${OUT}_ports.txt" "$NETWORK"

# Fase 3: Identificazione dispositivi OT con script NSE
echo "[3/4] Identificazione dispositivi OT..."
sudo nmap -sS -T2 --max-rate 20 -p 102 --script s7-info -iL "${OUT}_hosts.txt" -oA "${OUT}_s7.txt" "$NETWORK"
sudo nmap -sS -T2 --max-rate 20 -p 502 --script modbus-discover -iL "${OUT}_hosts.txt" -oA "${OUT}_modbus.txt" "$NETWORK"
sudo nmap -sU -T2 --max-rate 10 -p 47808 --script bacnet-info -iL "${OUT}_hosts.txt" -oA "${OUT}_bacnet.txt" "$NETWORK"

# Fase 4: Verifica porte IT non autorizzate su rete OT
echo "[4/4] Verifica porte IT su rete OT (anomalie)..."
# Porte IT che NON dovrebbero essere aperte su rete OT pura
FORBIDDEN_PORTS="135,139,445,1433,3306,5432,6379,27017"
sudo nmap -sS -T2 --open -p "$FORBIDDEN_PORTS" -iL "${OUT}_hosts.txt" -oA "${OUT}_anomalie.txt" "$NETWORK"

# Riepilogo
echo ""
echo "=== RIEPILOGO OT ASSET INVENTORY ==="
echo "Host trovati: $(wc -l < "${OUT}_hosts.txt")"
echo "Con Siemens S7: $(grep -c "S7" "${OUT}_s7.nmap" 2>/dev/null || echo 0)"
echo "Con Modbus: $(grep -c "Modbus" "${OUT}_modbus.nmap" 2>/dev/null || echo 0)"
echo "Con BACnet: $(grep -c "BACnet" "${OUT}_bacnet.nmap" 2>/dev/null || echo 0)"
echo "Anomalie IT su OT: $(grep -c "open" "${OUT}_anomalie.gnmap" 2>/dev/null || echo 0)"
echo ""
echo "Output: ${OUT}_*. {nmap,xml,gnmap}"
```

Regola OT	Dettaglio	Motivazione
Mai -T4 o -T5 in produzione	Usare sempre -T1 o -T2 su reti con stack TCP/IP limitati	Stack TCP/IP limitati possono andare in crash o freeze
Limite rate esplicito	--max-rate 30-50 su rete OT, 10-20 su rete IT	Dispositivi OT hanno buffer TCP ridotti
Testare prima in lab	Verificare su copia del sistema o in fase di manutenzione	Evitare downtime grave; fermare la produzione
Backup prima della scan	Snapshot configurazione PLC e HMI	Prevenzione perdita configurazione in caso di anomalia

Regola OT	Dettaglio	Motivazione
Scansione in orario di manutenzione	Pianificare scan durante fermate pianificate	Ridurre l'impatto su processo produttivo
Usare solo porte note OT	Evitare scan -p- su dispositivi OT	Porte alte sconosciute possono avere comportamento imprevedibile
Documentare ogni scan	Registrare data, scope, risultati, responsabilità	Consulenza IEC 62443, NIS2, ISO 27001
Coordinare con OT team	Mai agire senza consenso del responsabile OT	La sicurezza informatica non può essere considerata in isolamento; compromettere la sicurezza fisica

✓ **BEST PRACTICE** Per reti OT, considerare tool specializzati come Claroty, Dragos, o Nozomi per il monitoraggio continuo passivo (no active scanning). Nmap è ideale per scan periodici pianificati, non per monitoraggio continuo su OT.

Fonte: IEC 62443 (industrial cybersecurity standard); CISA ICS-CERT (cisa.gov/ics); ENISA Guidelines for Securing ICS (enisa.europa.eu); nmap.org/nsedoc/scripts/s7-info.html; nmap.org/nsedoc/scripts/modbus-discover.html